

Day 20 — Thu 24 Sep

Countdown: 2 days to General English & GK

Method: start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

● General — light revision

Source: General/01_GENERAL_PAPER_PLAYBOOK.md

1. Exam-day time allocation — the 180-minute plan

1.1 The recommended order of attempt

Do not attempt in the order printed on the paper. Attempt in this order:

Order	Component	Marks	Minutes	Cumulative
1st	Grammar / sentence construction	10	15	15
2nd	GK — objective portion	~20–25	25	40
3rd	GK — short descriptive portion	~25–30	35	75
4th	Amplification	10	20	95
5th	Précis	10	25	120
6th	Essay	20	50	170
—	Buffer / review / page numbering	—	10	180

1.2 Why this order

- **Grammar first (15 min).** These are the cheapest marks on the paper — pure recall, no thinking, no fatigue cost. Banking 8–9/10 in the first fifteen minutes settles your nerves and guarantees a floor. Never leave grammar until the end; if you run out of time you will lose marks you already knew.
- **GK second, while your memory is freshest.** Recall degrades under fatigue far faster than writing fluency does. Do the objective block fast — if you do not know an item in 20 seconds, mark it, move on, come back in the buffer.
- **Amplification and précis in the middle.** Both are *bounded* tasks — there is a correct-ish answer and a fixed length. They do not expand to fill available time the way an essay does. Doing them before the essay stops the essay from eating them.
- **Essay last, with a hard 50-minute block.** The essay is the one task where you can keep writing forever. Putting it last means the clock, not your judgement, ends it — and a 50-minute essay written at the end still scores 12–14 if you follow the skeleton. A rushed 15-minute précis at the end scores 4.

One exception: if the essay topic list is on page 1 and you can see it, *read the essay topics in the first 60 seconds* and pick your topic. Then let it marinate in the back of your mind while you do everything else. You will start the essay with your argument already half-formed. This single trick is worth 2–3 marks.

1.3 Micro-timings inside each block

Grammar (15 min) — 45–60 seconds per item. Do not agonise. If two options both "sound right", pick the shorter/simpler one and move on.

GK objective (25 min) — Two passes. Pass 1 (18 min): answer everything you know instantly. Pass 2 (7 min): return to the marked ones and make an educated guess.

There is no stated negative marking on this paper (negative marking is specified only for the Technical MCQ paper) — so **attempt every question**. Leave nothing blank.

GK short descriptive (35 min) — Budget by marks: roughly 5 minutes per 5-mark question, 3 minutes per 3-mark. Write to length, then stop.

Amplification (20 min) — 3 min planning, 15 min writing, 2 min checking.

Précis (25 min) — 6 min reading and marking the passage, 4 min drafting in the margin, 12 min writing the fair copy, 3 min counting words and writing the title.

Essay (50 min) — **8 min planning on rough paper**, 38 min writing, 4 min reading back. The 8 minutes of planning is non-negotiable. An unplanned essay loses more than 8 minutes' worth of marks in repetition and drift.

1.4 The three clock checkpoints

Write these on your rough sheet the moment you get it:

```
10:00 → Grammar done, GK objective underway
11:15 → ALL GK done. Nothing else touched.
11:40 → Amplification + précis done. Essay starts NOW.
12:00 → Pens down.
```

If at 11:40 you are not on the essay, **abandon whatever you are doing and start the essay**. Twenty marks unattempted is catastrophic; a half-finished précis is not.

● Core — SQL & transactions

Source: `_Shared/CORE_06_DBMS.md`

11. SQL — Structured Query Language

Concept

SQL is the standard **declarative** language for relational databases — you state *what* you want, the DBMS decides *how*. It is (mostly) **case-insensitive** for keywords, ends statements with `;`, and is grouped into sub-languages by purpose. Getting the four categories right is a guaranteed MCQ.

Sub-language	Purpose	Commands
DDL — Data Definition	Define/alter the structure (schema)	<code>CREATE</code> , <code>ALTER</code> , <code>DROP</code> , <code>TRUNCATE</code> , <code>RENAME</code>
DML — Data Manipulation	Work with the data (rows)	<code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>
DCL — Data Control	Permissions	<code>GRANT</code> , <code>REVOKE</code>
TCL — Transaction Control	Transaction boundaries	<code>COMMIT</code> , <code>ROLLBACK</code> , <code>SAVEPOINT</code> , <code>SET TRANSACTION</code>

(Some texts count `SELECT` as a separate DQL — Data Query Language. `TRUNCATE/DROP` are DDL and are auto-committed, which is why they cannot be rolled back — a favourite trap.)

DDL by example

```
CREATE TABLE STUDENT (  
  roll_no    INT          PRIMARY KEY,  
  name       VARCHAR(50)  NOT NULL,  
  dept       VARCHAR(20)  DEFAULT 'CSE',  
  cgpa       DECIMAL(3,2) CHECK (cgpa BETWEEN 0 AND 10),  
  dept_id    INT,  
  FOREIGN KEY (dept_id) REFERENCES DEPT(dept_id) ON DELETE SET NULL  
);  
  
ALTER TABLE STUDENT ADD email VARCHAR(60) UNIQUE;  -- add a column  
ALTER TABLE STUDENT DROP COLUMN email;             -- remove it  
DROP TABLE STUDENT;                                -- delete the whole table + data + structure  
TRUNCATE TABLE STUDENT;                             -- delete all rows, keep structure (fast, no rollback)
```

DELETE vs TRUNCATE vs DROP — a standard comparison:

	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Removes	Selected rows (with WHERE)	All rows	Table itself (structure + data)
WHERE clause	Yes	No	No
Rollback	Yes (logged)	Usually no (auto-commit)	No
Speed	Slow (row by row)	Fast	Fast
Resets identity	No	Yes	N/A

DML by example

```
INSERT INTO STUDENT (roll_no, name, dept) VALUES (1, 'Ravi', 'CSE');  
UPDATE STUDENT SET cgpa = 8.5 WHERE roll_no = 1;  
DELETE FROM STUDENT WHERE cgpa < 4.0;  
SELECT name, cgpa FROM STUDENT WHERE dept = 'CSE' ORDER BY cgpa DESC;
```

The **SELECT** skeleton and its **logical evaluation order** (worth stating, because it explains why you cannot use a column alias or an aggregate in **WHERE**):

```
SELECT  [DISTINCT] columns / aggregates      -- 5  
FROM    tables / joins                      -- 1  
WHERE   row filter (before grouping)        -- 2  
GROUP BY grouping columns                  -- 3  
HAVING  group filter (after grouping)       -- 4  
ORDER BY sort                             -- 6  
LIMIT   n                                  -- 7
```

Joins — every type with example and output

Two tables:

EMP			DEPT	
eid	name	dept_id	dept_id	dname
1	Ravi	10	10	Sales
2	Sita	20	20	IT
3	Anil	NULL	30	HR

INNER JOIN — only matching rows from both:

```
SELECT e.name, d.dname
FROM EMP e INNER JOIN DEPT d ON e.dept_id = d.dept_id;
```

name	dname
Ravi	Sales
Sita	IT

(Anil dropped — no match; HR dropped — no employee.)

LEFT (OUTER) JOIN — all left rows, NULLs where no right match:

```
SELECT e.name, d.dname FROM EMP e LEFT JOIN DEPT d ON e.dept_id = d.dept_id;
```

name	dname
Ravi	Sales
Sita	IT
Anil	NULL

RIGHT (OUTER) JOIN — all right rows, NULLs where no left match:

```
| name | dname |
|---|---|
| Ravi | Sales |
| Sita | IT |
| NULL | HR |
```

FULL OUTER JOIN — all rows from both sides, NULLs where unmatched:

```
| name | dname |
|---|---|
| Ravi | Sales |
| Sita | IT |
| Anil | NULL |
| NULL | HR |
```

CROSS JOIN — Cartesian product: every EMP row × every DEPT row (3 × 3 = 9 rows).

SELF JOIN — a table joined to itself (manager/employee):

```
SELECT e.name AS emp, m.name AS manager
FROM EMP e JOIN EMP m ON e.mgr_id = m.eid;
```

Join summary:

Join	Returns
INNER	Only rows matching in both
LEFT	All left + matching right (NULL if none)
RIGHT	All right + matching left (NULL if none)
FULL OUTER	All rows from both, matched where possible
CROSS	Cartesian product (all combinations)
SELF	A table joined to itself

Aggregates, GROUP BY and HAVING

Aggregate functions: **COUNT, SUM, AVG, MIN, MAX**. They collapse many rows into one value and **ignore NULLs** (except `COUNT(*)`, which counts rows).

```
SELECT dept_id, COUNT(*) AS headcount, AVG(salary) AS avg_sal
FROM EMP
WHERE salary IS NOT NULL           -- filter ROWS before grouping
GROUP BY dept_id
HAVING AVG(salary) > 50000         -- filter GROUPS after aggregating
ORDER BY avg_sal DESC;
```

WHERE vs HAVING — the single most examined SQL distinction: **WHERE filters individual rows before grouping and cannot contain aggregates; HAVING filters groups after aggregation and can.** If there is no `GROUP BY`, `HAVING` treats the whole result as one group.

Subqueries

A query nested inside another. Types:

- **Scalar** — returns one value: `WHERE salary > (SELECT AVG(salary) FROM EMP)`.
- **Multi-row** — with `IN, ANY, ALL`: `WHERE dept_id IN (SELECT dept_id FROM DEPT WHERE location='Kohima')`.
- **Correlated** — inner query references the outer row, re-evaluated per row:

```
SELECT name FROM EMP e
WHERE salary > (SELECT AVG(salary) FROM EMP WHERE dept_id = e.dept_id);
-- employees earning above their own department's average
```

- **EXISTS** — true if the subquery returns any row.

Views

A **view** is a **virtual table** defined by a stored query; it holds no data of its own but presents the result of its query on demand.

```
CREATE VIEW cse_toppers AS
SELECT roll_no, name, cgpa FROM STUDENT WHERE dept='CSE' AND cgpa > 8;
```

Uses: **security** (expose only some columns/rows), **simplicity** (hide complex joins), **logical data independence**. Limitations: a view is generally **updatable only if it maps to one base table without aggregates, DISTINCT, GROUP BY or joins**; complex views are read-only. A **materialised view** does store the result and must be refreshed.

Indexes

An **index** is an auxiliary structure (usually a **B+ tree**, sometimes a **hash**) that speeds up **retrieval** on the indexed column(s), at the cost of **extra storage** and **slower inserts/updates** (the index must be maintained).

```
CREATE INDEX idx_dept ON EMP(dept_id);
CREATE UNIQUE INDEX idx_email ON STUDENT(email);
```

A **primary/clustered index** determines the **physical order** of rows (one per table); a **secondary/non-clustered index** is a separate structure with pointers (many allowed). Rule of thumb: **index columns used in WHERE/JOIN/ORDER BY; do not over-index write-heavy tables.**

Stored procedures and functions

A **stored procedure** is a **named, precompiled block of SQL** stored in the database and

invoked by name — reducing network traffic, centralising logic and improving security (grant **EXECUTE** instead of table access).

```
CREATE PROCEDURE raise_salary(IN p_eid INT, IN p_pct DECIMAL(4,2))
BEGIN
    UPDATE EMP SET salary = salary * (1 + p_pct/100) WHERE eid = p_eid;
END;
CALL raise_salary(1, 10);    -- give employee 1 a 10% raise
```

A **function** is similar but **returns a value** and is usable **inside a query**; a procedure performs actions and is **called**. Both may take parameters (**IN**, **OUT**, **INOUT**).

Triggers

A **trigger** is a procedure that fires **automatically** in response to a data event (**INSERT**, **UPDATE**, **DELETE**) on a table — the database's event handler. Timing is **BEFORE** or **AFTER**, granularity is **FOR EACH ROW** (row-level) or statement-level.

```
CREATE TRIGGER audit_salary
AFTER UPDATE ON EMP
FOR EACH ROW
BEGIN
    INSERT INTO salary_log(eid, old_sal, new_sal, changed_at)
    VALUES (OLD.eid, OLD.salary, NEW.salary, NOW());
END;
```

OLD = the row before, **NEW** = the row after. Uses: **auditing**, enforcing complex business rules, maintaining derived/summary data, referential actions. Overuse hurts performance and hides logic — mention that as a disadvantage.

Likely exam questions

- **[5]** Differentiate between DDL, DML, DCL and TCL commands.
- **[5]** Differentiate between DELETE, TRUNCATE and DROP.
- **[5]** Differentiate between WHERE and HAVING clauses.
- **[5]** What is a view? State its advantages and when it is updatable.
- **[5]** What is a trigger? Explain BEFORE and AFTER triggers with an example.
- **[5]** Differentiate between a stored procedure and a function.
- **[10]** Explain all the join operations in SQL with example queries and their output.
- **[10]** Explain aggregate functions with GROUP BY and HAVING, and write SQL for a set of given queries.
- **[15]** Explain the categories of SQL. Write and explain queries demonstrating joins, subqueries, aggregates, views, indexes, a stored procedure and a trigger.

MCQ traps

Trap	Correct answer
" CREATE , ALTER , DROP belong to?"	DDL .
" GRANT / REVOKE are?"	DCL .
" COMMIT / ROLLBACK are?"	TCL .
"Which cannot be rolled back?"	TRUNCATE / DROP (DDL, auto-commit).
"WHERE can contain an aggregate?"	No — use HAVING .

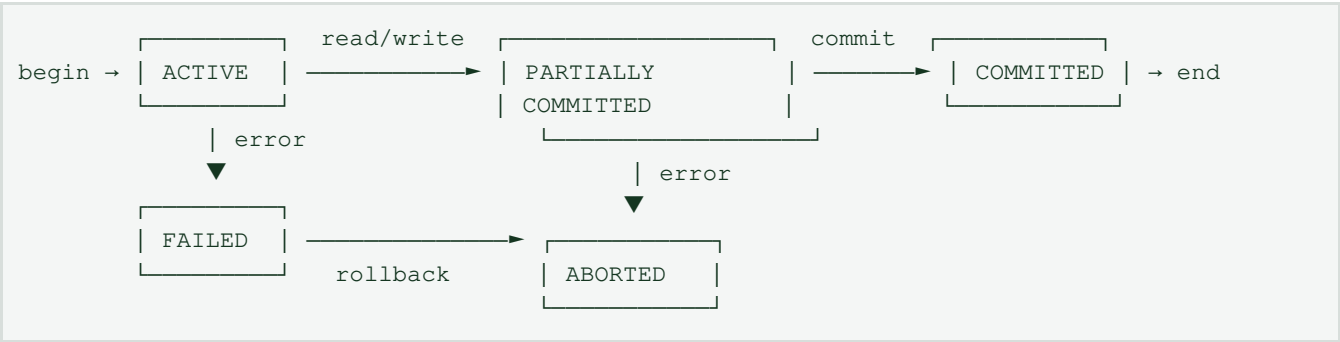
"A view stores data?"	No (a materialised view does).
"INNER JOIN returns?"	Only matching rows from both tables.
"LEFT JOIN keeps?"	All left-table rows.
"Which fires automatically on INSERT/UPDATE/DELETE?"	A trigger .
"A function differs from a procedure by?"	It returns a value and is usable in a query.
"COUNT(*) vs COUNT(col)?"	<code>COUNT (*)</code> counts rows; <code>COUNT (col)</code> ignores NULLs .

12. Transactions and ACID

Concept

A **transaction** is a **logical unit of work** — a sequence of operations that must be treated as a **single indivisible action**: either **all** of it happens, or **none** of it does. The canonical example is a bank transfer of ₹100 from A to B: debit A, credit B. If the system crashes after the debit but before the credit, ₹100 vanishes. The transaction concept forbids that partial state.

Transaction states: *Active* → *Partially committed* (last operation done) → **Committed** (changes made permanent) — or from Active/Partially committed → *Failed* → **Aborted** (rolled back). Draw this as a small state diagram; it is often worth 3–4 marks.



The ACID properties — the core 5/10-marker

Property	Guarantee	Ensured by
Atomicity	All-or-nothing — a transaction's operations either all complete or none do	Recovery manager (undo via the log)
Consistency	A transaction takes the database from one valid state to another , preserving all constraints	Application logic + integrity constraints
Isolation	Concurrent transactions do not interfere; each behaves as if it ran alone	Concurrency control (locking, timestamps)
Durability	Once committed , changes survive any subsequent failure	Recovery manager (write-ahead log, flush to disk)

Mnemonic: **A**tomicity, **C**onsistency, **I**solation, **D**urability. Pair each with its enforcer — that pairing is what turns a definition into a full answer.

Concurrency problems (why isolation matters)

Without isolation, interleaved transactions cause anomalies:

Problem	What happens
Lost update	Two transactions read the same value and both write; the second overwrites the first's update — one update is lost
Dirty read (uncommitted dependency)	T2 reads a value T1 wrote, then T1 aborts — T2 used data that never officially existed
Unrepeatable read	T1 reads a row twice and gets different values because T2 updated it in between
Phantom read	T1 re-runs a range query and finds new rows inserted by T2

SQL isolation levels trade correctness for concurrency:

Level	Prevents
READ UNCOMMITTED	Nothing (allows dirty reads)
READ COMMITTED	Dirty reads
REPEATABLE READ	Dirty + unrepeatable reads
SERIALIZABLE	All, including phantoms — behaves as if transactions ran one at a time

Likely exam questions

- **[5]** What is a transaction? Explain its states with a diagram.
- **[5]** Explain the ACID properties of a transaction.
- **[5]** Explain the concurrency problems: lost update, dirty read and unrepeatable read.
- **[10]** What is a transaction? Explain the ACID properties, stating how each is enforced, and describe the transaction state diagram.

MCQ traps

Trap	Correct answer
"'All-or-nothing' is which property?"	Atomicity .
"Committed changes survive a crash — which property?"	Durability .
"Isolation is ensured by?"	Concurrency control .
"Reading uncommitted data that is later rolled back is a?"	Dirty read .
"Highest isolation level?"	SERIALIZABLE .
"A transaction is a logical unit of?"	Work .

14. Concurrency control

Concept

Concurrency control is the mechanism that **produces only serializable, recoverable schedules** while still allowing interleaving. The two main families are **locking** (pessimistic) and **timestamp ordering** (optimistic about order).

Lock-based protocols

A **lock** is a privilege a transaction acquires on a data item before accessing it.

Lock mode	Grants	Compatible with
Shared (S) — read lock	Reading	Other S locks (many readers)
Exclusive (X) — write lock	Reading and writing	Nothing — one writer, no readers

Lock compatibility matrix: S–S compatible; S–X, X–S, X–X **incompatible**. Simply locking is not enough to guarantee serializability — *when* you release locks matters, which is why we need 2PL.

Two-Phase Locking (2PL) — the central protocol

Rule: every transaction obtains all its locks **before releasing any** — so its lifetime splits into two phases:

1. **Growing phase** — locks are **acquired**, none released.
2. **Shrinking phase** — locks are **released**, none acquired.

The moment of the first release is the **lock point**. **2PL guarantees conflict serializability**. Variants:

Variant	Rule	Solves
Basic 2PL	Growing then shrinking	Serializability, but risks deadlock and cascading rollback
Strict 2PL	Hold all exclusive (write) locks until commit/abort	Prevents cascading rollback ; recoverable
Rigorous 2PL	Hold all locks (S and X) until commit/abort	Simplest; serial order = commit order
Conservative (static) 2PL	Acquire all locks before starting	Deadlock-free (but needs to predeclare, low concurrency)

Deadlock: two transactions each hold a lock the other needs, waiting forever. Handling:

- **Prevention** — timestamp schemes **wait-die** (older waits, younger dies/aborts) and **wound-wait** (older wounds/aborts younger, younger waits).
- **Avoidance** — conservative 2PL.
- **Detection** — build a **wait-for graph**; a **cycle = deadlock**; abort a victim.
- **Timeout** — abort a transaction that waits too long.

Timestamp ordering (TO)

Each transaction gets a unique **timestamp TS(T)** at start (older = smaller). Each data item keeps a **read-timestamp** and **write-timestamp**. The protocol allows an operation only if it respects timestamp order; otherwise the offending transaction is **aborted and restarted with a new timestamp**. It **orders transactions by their start time**, produces serializable schedules, and is **deadlock-free** (no waiting), but can cause **starvation** (a transaction repeatedly restarted). The **Thomas Write Rule** optimises it by ignoring obsolete writes.

Multiversion concurrency control (MVCC) keeps multiple versions so readers never block writers (used by PostgreSQL, Oracle); **optimistic concurrency control** validates only at commit, best when conflicts are rare.

	Locking (2PL)	Timestamp ordering
Style	Pessimistic — block until safe	Order by start time, abort if violated

Deadlock	Possible	Impossible (no waiting)
Starvation	Possible	Possible (repeated restarts)
Serial order	By lock point	By timestamp

Likely exam questions

- [5] What are shared and exclusive locks? Give the lock-compatibility matrix.
- [5] Explain the two phases of the 2PL protocol. What does it guarantee?
- [5] Differentiate between strict 2PL and rigorous 2PL.
- [5] What is a deadlock? Explain wait-die and wound-wait.
- [10] Explain lock-based concurrency control and the 2PL protocol with its variants, and discuss deadlock handling.
- [10] Compare lock-based and timestamp-based concurrency control.

MCQ traps

Trap	Correct answer
"2PL guarantees?"	Conflict serializability .
"In the growing phase a transaction may?"	Only acquire locks.
"Which lock is compatible with another shared lock?"	Shared (S) .
"Strict 2PL holds which locks till commit?"	Exclusive (write) locks.
"Which protocol is deadlock-free?"	Timestamp ordering (and conservative 2PL).
"Cycle in the wait-for graph means?"	Deadlock .
"Older transaction waits, younger aborts — this is?"	Wait-die .

15. Database recovery

Concept

Recovery restores the database to the **most recent consistent state** after a failure (transaction error, system crash, disk failure). Its job is to enforce **atomicity** (undo partial transactions) and **durability** (redo committed ones). The central tool is the **transaction log** written **before** the data — the **Write-Ahead Logging (WAL)** principle.

The WAL rule (state it): the **log record describing a change must reach stable storage before the changed data page does**, and all log records of a transaction must be flushed before it commits. Without WAL there would be nothing to undo/redo a crash with.

A log record looks like `<T, X, old_value, new_value>` (plus `<T start>`, `<T commit>`, `<T abort>`).

Log-based recovery techniques

Technique	When data is written to disk	On recovery
Deferred update (NO-UNDO/REDO)	Only after commit — changes buffered until then	REDO committed transactions; uncommitted ones never touched the DB, so no undo needed

Immediate update (UNDO/REDO)	Changes may be written before commit	UNDO uncommitted transactions (restore old values) and REDO committed ones (apply new values)
--	--	--

The rule for immediate update: **undo in reverse order, redo in forward order**. A transaction with a `<T commit>` in the log → **redo**; one without → **undo**.

Checkpoints

Redoing the *entire* log after every crash is impossibly slow. A **checkpoint** is a point at which the DBMS **flushes all buffers to disk and writes a `<checkpoint>` record**, so recovery need only consider transactions active **at or after** the last checkpoint. On restart, scan back to the last checkpoint and build two lists: **REDO** (committed after the checkpoint) and **UNDO** (active but not committed). Everything before the checkpoint is already safely on disk.

ARIES is the industrial algorithm — three phases: **Analysis** (find the dirty pages and active transactions from the last checkpoint), **Redo** (repeat history to reconstruct state), **Undo** (roll back losers), using an **LSN** (log sequence number) per record. Naming the three phases is enough for the exam.

Shadow paging is an alternative that keeps a **shadow (old) copy** of the page table; commit atomically switches to the new page table, and recovery just reverts to the shadow — no log, but poor concurrency and fragmentation.

Types of failure

Failure	Recovery
Transaction failure (logical error, deadlock abort)	Roll back that transaction (undo)
System crash (power/OS/DBMS)	Log-based redo/undo from last checkpoint
Media/disk failure	Restore from backup + roll forward the archived log

Likely exam questions

- **[5]** State the Write-Ahead Logging rule. Why is it necessary?
- **[5]** Differentiate between deferred and immediate database modification.
- **[5]** What is a checkpoint? How does it speed up recovery?
- **[10]** Explain log-based recovery. Describe deferred and immediate update with the undo/redo actions and the role of checkpoints.

MCQ traps

Trap	Correct answer
"The log is written before the data — this is?"	Write-Ahead Logging (WAL) .
"Deferred update needs which operation on recovery?"	REDO only (no undo).
"Immediate update needs?"	UNDO and REDO .
"A transaction with no commit record is?"	Undone .
"Checkpoints reduce?"	The amount of log to redo/undo on recovery.
"ARIES phases?"	Analysis, Redo, Undo .
"Which recovery method uses no log?"	Shadow paging .

16. Database security, authorization and SQL injection

Concept

Database security protects the database against three threats: **loss of confidentiality** (unauthorised disclosure), **loss of integrity** (improper modification) and **loss of availability** (denial of access). It rests on **authentication** (proving *who* you are), **authorization** (deciding *what* you may do), **access control** and **auditing**.

Term	Meaning
Authentication	Verifying identity — login/password, certificates, biometrics
Authorization	Granting rights (privileges) to authenticated users
Access control	Enforcing those rights — DAC, MAC, RBAC (see below)
Auditing	Logging who did what, for accountability and forensics
Encryption	Protecting data at rest and in transit (TDE, column encryption)

Access-control models:

- **DAC** (Discretionary) — the object's owner grants/revokes privileges (SQL's **GRANT/REVOKE**).
- **MAC** (Mandatory) — system-enforced security labels/clearances (Bell-LaPadula); used in high-security/government systems.
- **RBAC** (Role-Based) — privileges granted to **roles**, users assigned to roles; the practical enterprise model.

Authorization in SQL (DCL):

```
GRANT SELECT, INSERT ON STUDENT TO clerk;           -- give privileges
GRANT SELECT ON STUDENT TO analyst WITH GRANT OPTION; -- allow re-granting
REVOKE INSERT ON STUDENT FROM clerk;                -- take them back
CREATE ROLE clerk; GRANT clerk TO ravi;              -- role-based
```

Views as a security tool: grant access to a view exposing only permitted rows/columns instead of the base table — **value-independent** (columns) and **value-dependent** (rows via WHERE) authorization.

SQL injection — the named forensic topic

SQL injection is an attack that inserts malicious SQL through **unsanitised user input**, tricking the application into executing unintended commands. It is the classic web-application database vulnerability and is named explicitly in your Forensic syllabus.

How it works. An application builds a query by **string concatenation**:

```
"SELECT * FROM users WHERE name='" + input + "' AND pass='" + pw + '"
```

If the attacker enters username `' OR '1'='1' --`, the query becomes:

```
SELECT * FROM users WHERE name='' OR '1'='1' --' AND pass='...'
```

`'1'='1'` is always true and `--` comments out the password check, so the attacker logs in without a password. Worse payloads use **UNION SELECT** to steal other tables, or stacked queries like `'; DROP TABLE users; --`.

Types: in-band (classic/error-based, UNION-based), **blind** (boolean- or time-based, when no

output is shown), and out-of-band.

Prevention — the marks are in this list:

1. **Parameterised queries / prepared statements** (bind variables) — the **single best defence**; data can never be parsed as code.
2. **Stored procedures** with parameters (if they don't themselves build dynamic SQL).
3. **Input validation and sanitisation** — whitelist allowed characters, escape special ones.
4. **Least-privilege** database accounts — the web app should not run as DBA.
5. **Web application firewall (WAF)** and error-message suppression (don't leak schema).

Parameterised example (safe):

```
PREPARE stmt FROM 'SELECT * FROM users WHERE name=? AND pass=?';  
-- the ? placeholders are bound to values, never concatenated as SQL text
```

Other DB security concerns

- **Statistical database security** — aggregate queries can leak individual data (**inference**); mitigated by query restriction and noise.
- **Data dictionary / catalog** — must itself be protected; it reveals the whole schema.
- **Auditing and logging** for non-repudiation and forensic reconstruction.

Likely exam questions

- **[5]** Differentiate between authentication and authorization.
- **[5]** Explain DAC, MAC and RBAC.
- **[5]** Explain GRANT and REVOKE with the WITH GRANT OPTION.
- **[5]** How can views be used to enforce security?
- **[5]** What is SQL injection? Give one example and two prevention techniques.
- **[10]** What is SQL injection? Explain how it works with an example, its types, and the measures to prevent it.
- **[10]** Explain database security: authentication, authorization, access-control models and the use of GRANT/REVOKE and views.

MCQ traps

Trap	Correct answer
"Proving identity is?"	Authentication . Deciding rights = authorization .
"GRANT/REVOKE implement which access model?"	DAC (discretionary).
"Best defence against SQL injection?"	Parameterised / prepared statements .
" <code>' OR '1'='1' --</code> is an example of?"	SQL injection .
"Privileges granted to roles, not users, is?"	RBAC .
"WITH GRANT OPTION lets the grantee?"	Re-grant the privilege .

● Core — UNIX / Linux commands

Source: `_Shared/CORE_04_UnixLinux.md`

CORE 04 — UNIX / Linux

Shared-core file 4 of the NPSC CTSE 2026 prep set.

Read with `OFFICIAL_SYLLABUS.md` open. Everything here is written to be revised at three depths: **one-line fact** (MCQ), **5-mark answer** (~5–6 crisp points), **10/15-mark answer** (worked explanation + hand-drawn diagram).

0. Why this matters — where UNIX appears in your three papers

Elective	Where it appears	How heavy
CS Diploma	Paper-II §2 — an <i>entire numbered unit</i> called "UNIX Operating System". Names ~30 commands explicitly by name .	Very heavy. This is the single most predictable unit in your whole Diploma paper.
Computer Forensic	TP-I §8 (Operating Systems → "UNIX: The Unix System: File system, process management, shell variables, command line programming. Filters and Commands") and TP-I §5 ("Linux System and Artifacts: Linux file system — Ownership and permissions, hidden files, User Accounts and Logs")	Heavy , and note the forensic slant: permissions, hidden files, logs.
CS Degree	Not named. But OS Paper-II §1 (file system management, process management, security & protection) is answered <i>beautifully</i> using UNIX as the worked example.	Indirect — use it as illustration.

The strategic read. The Diploma syllabus lists commands by name. When a syllabus names a command, an examiner can write a one-mark MCQ on it almost mechanically ("Which command counts lines, words and characters?"). So the quick-reference table in §7 is not filler — it *is* the MCQ paper. Learn it cold. Meanwhile the Forensic paper wants you to reason about *permissions and ownership*, which is where the `chmod` arithmetic in §5 earns its 10 marks.

Topic checklist

- [] UNIX history, development, and why it looks the way it does
- [] Features of UNIX (multiuser, multitasking, portability, hierarchical FS)
- [] Architecture: hardware → kernel → shell → utilities (draw the onion)
- [] Kernel functions; shell types (`sh`, `bash`, `csh`, `ksh`, `tcsh`, `zsh`)
- [] Hierarchical directory structure and the standard / subdirectories
- [] File types (all seven) and how to identify each from `ls -l`
- [] Inodes — what is stored *in* one and what is emphatically *not*
- [] Hard links vs symbolic links
- [] Ownership: user / group / other; `chown`, `chgrp`
- [] **Permissions — symbolic and numeric; `chmod` calculations both directions**
- [] `umask` and how it derives default permissions
- [] Special permission bits: SUID, SGID, sticky bit
- [] Navigation & file management commands
- [] File processing / filter commands (the big Diploma list)
- [] Formatting and printing — `pr` options, `lp`
- [] Online help — `man` sections, `help`, `info`, `whatis`, `apropos`

- [] Mathematical — `bc`, `expr`, `factor`, `units`
- [] Communication — `write`, `mail`, `wall`, `mesg`, `talk`
- [] System info — `who`, `who am i`, `tty`, `date`, `cal`, `uname`, `uptime`
- [] Booting procedure stages; `init` / `runlevels` / `systemd`; login process
- [] Startup and shutdown commands
- [] Shell variables (local, environment, special/positional)
- [] Shell scripting basics — shebang, conditionals, loops, functions
- [] Pipes, redirection, tee, filters
- [] Process management — `ps`, `kill`, signals, `&`, `nohup`, `jobs`, `fg`, `bg`, `nice`
- [] vi editor three modes

1. History and development of UNIX

Concept — the plain-English story first

In the mid-1960s Bell Labs, GE and MIT were building **Multics**, an enormously ambitious time-sharing operating system. It was late, huge and over-engineered, and Bell Labs pulled out in 1969. Two of the researchers left holding the bag, **Ken Thompson** and **Dennis Ritchie**, wanted the *nice parts* of Multics — a hierarchical file system, multiple users sharing one machine — without the bloat. Thompson found a cast-off DEC PDP-7 and wrote a tiny single-user system for it. Brian Kernighan punned that if Multics was "multiplexed", this thing was "uniplexed" — **UNICS** — later spelled **UNIX**.

Three decisions made in those first few years explain almost everything about UNIX today:

1. **In 1973 it was rewritten in C.** Every operating system before it was written in assembly, and assembly is machine-specific. Rewriting UNIX in a high-level language made it *portable* — you could move it to a new machine by writing a C compiler for that machine. This is the single most important fact in UNIX history, and it is a favourite one-mark MCQ.
2. **Everything is a file.** Disks, terminals, printers, even processes (via `/proc`) are presented through the same read/write interface. So the same handful of commands work on everything, and pipes work universally.
3. **Small tools, one job each, joined by pipes.** Rather than one giant program with fifty options, UNIX gives you `sort`, `grep`, `wc`, `cut` and lets you chain them. This is the "UNIX philosophy" and it is why the command list in §7 is long but each entry is short.

Timeline (memorise the bold rows — these are the MCQ rows)

Year	Event
1965	Multics project begins (Bell Labs + GE + MIT)
1969	Bell Labs withdraws. Ken Thompson writes first UNIX on a PDP-7 , in assembly
1970	Named "UNIX"; UNIX epoch begins 1 Jan 1970 00:00:00 UTC
1971	First Edition; ported to PDP-11; <code>roff</code> /troff added
1973	Rewritten in C by Dennis Ritchie & Ken Thompson → portability
1975	Sixth Edition released to universities (source included) — spreads UNIX

1977	BSD (Berkeley Software Distribution) branch begins at UC Berkeley
1979	Seventh Edition — Bourne shell (<code>sh</code>), the "ancestor" UNIX
1983	AT&T System V released — the other great branch. C shell (<code>csh</code>) by Bill Joy
1984	Richard Stallman starts GNU ("GNU's Not UNIX") project
1987	MINIX by Andrew Tanenbaum (teaching OS)
1988	POSIX (IEEE 1003) standard — portable interface across UNIX variants
1991	Linus Torvalds releases the Linux kernel (v0.01), Helsinki
1992	Linux relicensed under GPL ; GNU tools + Linux kernel = usable OS
1993	FreeBSD, NetBSD; Linux distributions begin (Slackware, Debian)
2001	Mac OS X — a BSD-derived (Darwin/Mach) UNIX, relevant to your Forensic paper

The two great families — you will be asked to distinguish them:

	System V (AT&T)	BSD (Berkeley)
Init	<code>/etc/inittab</code> , runlevels, <code>/etc/rc.d/</code> scripts	<code>/etc/rc</code> scripts, no runlevels
Print	<code>lp</code> , <code>lpstat</code> , <code>cancel</code>	<code>lpr</code> , <code>lpq</code> , <code>lprm</code>
Process listing	<code>ps -ef</code>	<code>ps aux</code>
Terminal	<code>termcap</code> replaced by <code>terminfo</code>	<code>termcap</code>
Descendants	Solaris, AIX, HP-UX	FreeBSD, NetBSD, OpenBSD, macOS

Linux is a **System V-style** userland with many BSD features, GNU utilities, and its own kernel written from scratch. Note carefully: **Linux is not derived from UNIX source code** — it is a UNIX-like system, written independently, that conforms to POSIX. Examiners like this distinction.

Features of UNIX — the standard 5-mark list

Feature	One-line explanation
Multuser	Many users log in simultaneously; each has own home dir, permissions, environment
Multitasking	Many processes run concurrently via time-sharing; supports background jobs
Portability	Written in C; ~95% of the kernel is machine-independent
Hierarchical file system	Single inverted-tree rooted at <code>/</code> ; all devices mounted into it
Everything is a file	Devices, pipes, sockets accessed via the same read/write calls
Security	User/group/other permissions, ownership, encrypted passwords, SUID model
Pipes & filters	Output of one command feeds directly into the next — tool composition
Shell as programmable interface	The command interpreter is also a full scripting language

Open / standardised	POSIX conformance; source availability in Linux/BSD
Communication	Built-in inter-user (<code>write</code> , <code>mail</code> , <code>wall</code>) and inter-process (pipes, signals, IPC)

Likely exam questions

- **[5]** Write a short note on the history and development of the UNIX operating system.
- **[5]** List and explain any five salient features of UNIX.
- **[5]** Distinguish between UNIX and Linux.
- **[10]** Trace the evolution of UNIX from Multics to Linux. Why was the 1973 rewrite in C the turning point? Compare the System V and BSD lineages.

MCQ traps

Trap	Correct answer
"Who developed UNIX?"	Ken Thompson (with Dennis Ritchie). Ritchie alone → C language.
"UNIX was originally written in?"	Assembly language (PDP-7). C came in 1973.
"First machine UNIX ran on?"	DEC PDP-7 (not PDP-11 — that was the 1971 port).
"Linux kernel author / year?"	Linus Torvalds, 1991 .
"UNIX epoch?"	1 January 1970 .
"Full form of UNIX/UNICS?"	Uniplexed Information and Computing Service/System.
"Is Linux a UNIX?"	It is UNIX-like / POSIX-compliant , not derived from UNIX code.
"GNU stands for?"	GNU's Not UNIX (a recursive acronym).

2. UNIX architecture — kernel, shell, utilities

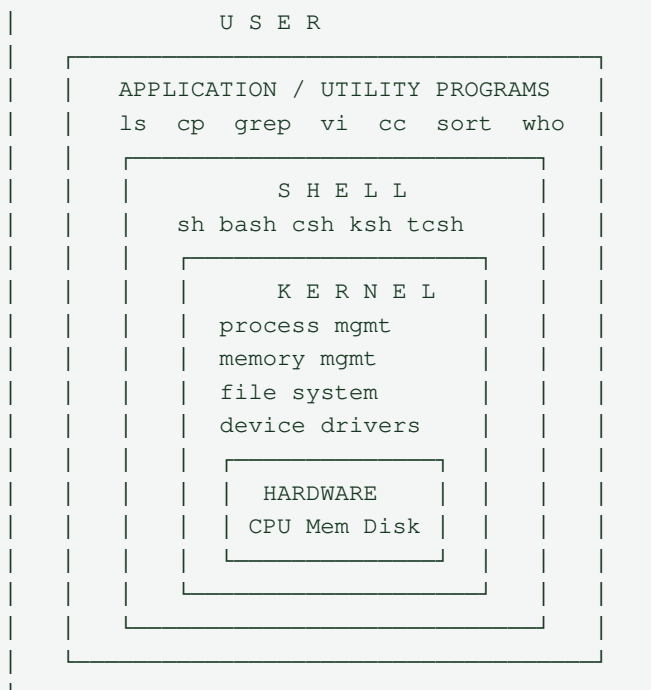
Concept

Think of UNIX as an **onion with four layers**. At the dead centre sits the **hardware**. Wrapped around it is the **kernel**, the only piece of software allowed to touch hardware directly. Around the kernel sits the **shell**, which is just an ordinary program whose job is to read what you type and ask the kernel to do it. On the outside sit the **application utilities** — `ls`, `grep`, `vi`, your own programs — and finally the **user**.

The crucial insight, and the one that gets marks: **the shell is not part of the kernel**. It is a replaceable user-level program. That is why you can have half a dozen different shells on the same machine and switch between them freely, while there is exactly one kernel. When you type `ls`, the shell does not list files — it locates the `/bin/ls` program, asks the kernel to create a process for it, and waits. The only way any program, shell included, gets work out of the kernel is through **system calls** (`open`, `read`, `write`, `fork`, `exec`, `wait`, `exit`).

Diagram to hand-draw — "the UNIX onion"

Draw **four concentric circles**. Label from the inside out:



Add an arrow from SHELL to KERNEL labelled "**system calls**" — that single arrow is worth a mark on its own, because it names the interface. Also mark that utilities can call the kernel directly too (they need not go through the shell).

Key points — the kernel

- Loaded into memory at boot and **stays resident** until shutdown; it is the core of the OS.
- Runs in **privileged / kernel mode**; user programs run in **user mode**.
- Everything it does is requested through **system calls** (~300 in Linux) — the boundary between user space and kernel space.
- Five classical responsibilities:

Kernel subsystem	Responsibility
Process management	Creating (<code>fork</code>), running (<code>exec</code>), scheduling, terminating processes; context switching; signals; IPC
Memory management	Allocating/freeing memory, virtual memory, paging, swapping, protection between processes
File system management	Directories, inodes, blocks, mounting, buffering, permissions enforcement
Device management	Device drivers; the abstraction that makes <code>/dev/sda</code> look like a file
Networking / system calls	Protocol stack, sockets; providing the system-call interface itself

- **Monolithic** kernel design in UNIX/Linux (all subsystems in one address space), as against microkernel designs (Mach, MINIX). Linux is monolithic but **modular** — drivers can be loaded/unloaded at runtime (`insmod`, `lsmod`, `rmmod`, `modprobe`).

Key points — the shell

- The **command interpreter**: reads a command line, parses it, expands wildcards and variables, handles redirection and pipes, then forks/execs the program.
- It is a **user program**, not part of the kernel — replaceable per user.
- It is also a **programming language** (variables, `if`, loops, functions) — hence shell scripts.
- The user's login shell is recorded in the **last field of `/etc/passwd`**.
- The shell's own read-execute cycle: **prompt** → **read** → **parse** → **expand** → **fork** → **exec** → **wait** → **prompt**.

Shell	Command	Author	Notes
Bourne shell	<code>sh</code>	Stephen Bourne, 1979	The original standard; default prompt <code>\$</code> ; best for scripting portability
C shell	<code>csh</code>	Bill Joy, BSD	C-like syntax; introduced history , aliases , job control ; prompt <code>%</code>
Korn shell	<code>ksh</code>	David Korn, AT&T	Bourne-compatible + csh features + arrays, arithmetic; prompt <code>\$</code>
Bourne-Again shell	<code>bash</code>	Brian Fox, GNU	Default on Linux; sh-compatible superset; command completion
TC shell	<code>tcsh</code>	—	Enhanced csh with editing/completion
Z shell	<code>zsh</code>	—	Superset of bash/ksh; default on modern macOS
Restricted shell	<code>rsh</code> / <code>rbash</code>	—	Cannot <code>cd</code> , change PATH, or redirect — used for locked-down accounts

Root's prompt is conventionally `#`; ordinary users get `$` (or `%` in csh). This is an MCQ.

Key points — utilities / application programs

- Ordinary executables living in `/bin`, `/usr/bin`, `/sbin`, `/usr/sbin`.
- Each does **one job**; composed with pipes.
- Categories: file management, filters, text processing, compilers, editors, networking, system administration.

Likely exam questions

- **[5]** With a neat diagram explain the structure of the UNIX operating system.
- **[5]** What is a shell? Name any four shells and state one distinguishing feature of each.
- **[5]** Differentiate between the kernel and the shell.
- **[10]** Explain the architecture of UNIX with a labelled diagram. Discuss in detail the functions of the kernel and describe how a typed command is executed end to end.
- **[15]** "UNIX is built in layers." Justify with a diagram, explaining hardware, kernel, shell and utilities, the role of system calls, and how the design delivers portability, multiuser operation and security.

MCQ traps

Trap	Correct answer
------	----------------

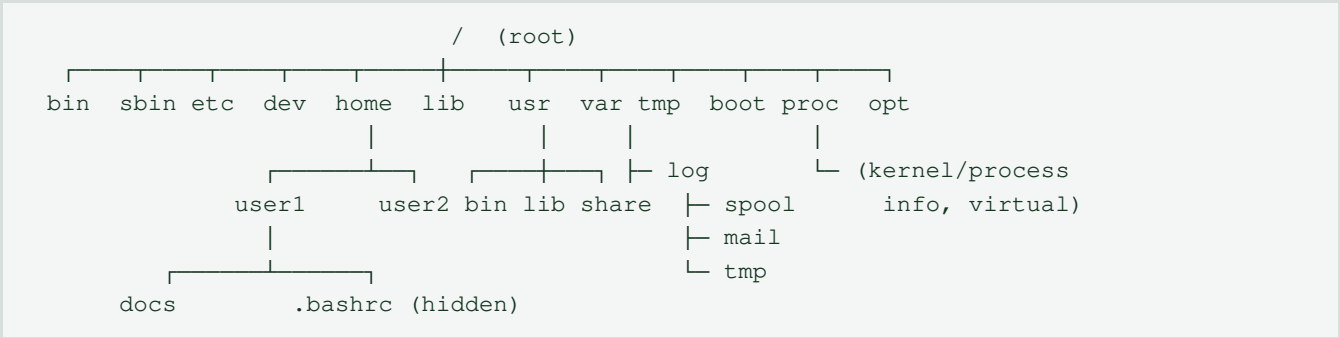
"The shell is a part of the kernel."	False — it is a user-level program.
"Which is the default shell in Linux?"	bash . Default on modern macOS is zsh . Original UNIX default is sh .
"Interface between user and kernel?"	Shell . Interface between shell and hardware → kernel.
"Which shell introduced job control and history?"	C shell (csh) .
"How many kernels can run?"	One . Many shells, one kernel.
"Login shell is stored in?"	/etc/passwd , last (7th) field.
"Linux kernel type?"	Monolithic (modular) — not microkernel.

3. The UNIX file system and hierarchical directory structure

Concept

DOS and Windows give each disk its own letter — **C:**, **D:** — so there are several separate trees. UNIX refuses to do this. There is exactly **one** tree, and its root is called **/** (pronounced "root", "slash"). A second disk does not get a letter; it gets **mounted** onto a directory inside the existing tree, say **/home**, and from then on it is simply part of the same tree. Users cannot tell where one physical device ends and another begins, which is exactly the point: the file system is a *logical* structure that hides physical devices. An **inverted tree** is the standard phrase — root at the top, branches downward.

Diagram to hand-draw — the standard hierarchy



The standard directories — learn this table

Directory	Holds
/	Root of the entire hierarchy; everything hangs off it
/bin	Essential user binaries available to all users: ls , cp , cat , mv
/sbin	System binaries for the superuser: fsck , init , shutdown , mkfs
/etc	Configuration files (text): passwd , shadow , group , fstab , inittab , hosts , profile . <i>Never binaries.</i>
/dev	Device files : /dev/sda (disk), /dev/tty (terminal), /dev/null (bit bucket), /dev/zero , /dev/random

<code>/home</code>	Ordinary users' home directories (<code>/home/ravi</code>)
<code>/root</code>	The superuser's home directory (not the same as <code>/</code>)
<code>/lib</code>	Shared libraries and kernel modules needed by <code>/bin</code> and <code>/sbin</code>
<code>/usr</code>	Secondary hierarchy of user programs — <code>/usr/bin</code> , <code>/usr/lib</code> , <code>/usr/share</code> , <code>/usr/local</code> , <code>/usr/include</code> . Read-only, shareable
<code>/var</code>	Variable data that changes constantly: <code>/var/log</code> , <code>/var/spool/mail</code> , <code>/var/spool/cron</code> , <code>/var/tmp</code>
<code>/tmp</code>	Temporary files; world-writable with sticky bit ; usually cleared at boot
<code>/boot</code>	Bootloader files and the kernel image (<code>vmlinuz</code>), <code>initrd</code> , GRUB config
<code>/proc</code>	Virtual/pseudo file system — a window on kernel and process state. Occupies zero disk ; created in memory. <code>/proc/cpuinfo</code> , <code>/proc/meminfo</code> , <code>/proc/<pid>/</code>
<code>/sys</code>	Virtual FS exposing kernel device/driver model (modern Linux)
<code>/mnt</code> , <code>/media</code>	Mount points for temporary / removable file systems
<code>/opt</code>	Optional, third-party add-on application packages
<code>/srv</code>	Data served by the system (web, ftp)
<code>/lost+found</code>	Files recovered by <code>fsck</code> after a crash; one per file system

For your Forensic paper, memorise these evidentiary locations:

Artefact	Path
User accounts	<code>/etc/passwd</code> (world-readable), <code>/etc/shadow</code> (hashes, root-only, mode 640/600)
Groups	<code>/etc/group</code> , <code>/etc/gshadow</code>
Successful/failed logins (binary)	<code>/var/log/wtmp</code> (read with <code>last</code>), <code>/var/log/btmp</code> (<code>lastb</code>), <code>/var/run/utmp</code> (<code>who</code>)
Last login per user	<code>/var/log/lastlog</code> (read with <code>lastlog</code>)
Auth events	<code>/var/log/auth.log</code> (Debian) or <code>/var/log/secure</code> (RHEL)
General system messages	<code>/var/log/messages</code> , <code>/var/log/syslog</code> , <code>dmesg</code>
Shell history	<code>~/.bash_history</code>
Scheduled tasks	<code>/var/spool/cron/crontabs/<user></code> , <code>/etc/crontab</code> , <code>/etc/cron.d/</code>
Mounted FS at boot	<code>/etc/fstab</code> ; currently mounted → <code>/etc/mtab</code> , <code>mount</code> , <code>df</code>

File system internal structure — the four blocks

A UNIX disk partition is divided into four regions. Draw this as a **horizontal bar split into four labelled segments** — it is one of the easiest diagram marks available.

BOOT BLOCK	SUPER BLOCK	INODE BLOCK	DATA BLOCK
block 0		(inode table)	(file contents + dirs)

Region	Contents
Boot block	First block (block 0) of the file system. Holds the bootstrap loader that pulls the kernel into memory. Only the boot block of the root file system is actually used; others are empty but reserved.
Super block	The file system's "table of contents": size of the FS, size of the inode list, number of free blocks and free inodes, list of free blocks/inodes, FS state (clean/dirty), block size, magic number. Corrupt superblock = unusable file system , hence backup superblocks are kept.
Inode block	The inode table — a fixed-size array of inodes, one per file, created when the FS is made. This is why a file system can run out of inodes while still having free space (millions of tiny files).
Data block	Actual file contents, and directory entries. Numbered from the end of the inode list to the end of the FS.

Common UNIX/Linux file system types: **ext2 / ext3 / ext4** (ext3 and ext4 are journaling), **XFS, Btrfs, UFS/FFS** (BSD), **JFS, ReiserFS, ZFS**, plus **swap**. Journaling means metadata changes are written to a log first so a crash can be recovered quickly without a full **fsck** — a good one-liner for the Forensic paper too.

File types — all seven

ls -l prints a ten-character string. The **first character is the file type**; the remaining nine are permissions. Memorise the seven type characters:

Char	Type	Explanation	Example
-	Ordinary / regular file	Text, binary, image, anything	<code>report.txt</code> , <code>/bin/ls</code>
d	Directory	A file containing (filename, inode number) pairs	<code>/home</code>
l	Symbolic (soft) link	A file whose content is a <i>pathname</i> pointing elsewhere	<code>/bin → /usr/bin</code>
c	Character special file	Device transferring data character by character, unbuffered	<code>/dev/tty</code> , <code>/dev/null</code> , terminals, printers
b	Block special file	Device transferring data in fixed-size blocks, buffered	<code>/dev/sda</code> , hard disks, CD-ROM
p	Named pipe (FIFO)	IPC channel with a name in the file system (<code>mkfifo</code>)	<code>/tmp/mypipe</code>
s	Socket	Endpoint for bidirectional IPC, often network	<code>/var/run/docker.sock</code>

Broad classification often asked as a 5-marker: **(1) Ordinary files, (2) Directory files, (3) Device/special files** — with links, pipes and sockets as further special types.

Use **file <name>** to determine content type (it reads magic numbers, not the extension) — worth remembering for the Forensic paper, because **UNIX has no concept of file extensions**; `report.txt` and `report` are equally valid names and the `.txt` means nothing to the kernel.

Hidden files: any file whose name begins with a **dot** (`.bashrc`, `.ssh`, `.profile`) is hidden from a plain `ls`. Reveal with `ls -a`. This is explicitly in your Forensic syllabus ("hidden files") — the exam point is that hiding is a *naming convention honoured by `ls`*, not a file attribute or a security feature. Note `.` (current directory) and `..` (parent directory) are themselves entries in every directory; `ls -A` shows hidden files but suppresses `.` and `..`.

Likely exam questions

- **[5]** Explain the hierarchical directory structure of UNIX with a diagram.
- **[5]** List and explain the different types of files in UNIX.
- **[5]** Explain the structure of a UNIX file system (boot block, super block, inode block, data block).
- **[5]** What are hidden files? How are they created and displayed?
- **[10]** Describe the UNIX file system in detail: the standard directory hierarchy, the four regions of a disk partition, and the seven file types with examples.
- **[15]** Explain the UNIX file system architecture. Include the directory hierarchy, the role of the superblock and inode table, file types, and describe how the kernel resolves the pathname `/home/ravi/notes.txt` to actual disk blocks.

MCQ traps

Trap	Correct answer
"Which directory holds configuration files?"	<code>/etc</code> (not <code>/config</code> , not <code>/var</code>).
"Which holds device files?"	<code>/dev</code> .
"Which is a virtual file system occupying no disk?"	<code>/proc</code> .
"Root user's home directory?"	<code>/root</code> , not <code>/</code> .
"Which block holds free block list and FS size?"	Super block (not boot block).
"Log files live in?"	<code>/var/log</code> .
"Which command shows hidden files?"	<code>ls -a</code> .
"First character <code>b</code> in <code>ls -l</code> means?"	Block special file (<code>c</code> = character special).
"Number of file systems trees in UNIX?"	One — a single tree rooted at <code>/</code> .
"Password hashes are in?"	<code>/etc/shadow</code> , not <code>/etc/passwd</code> .

4. Inodes and links

Concept

Here is the idea that surprises everybody: **in UNIX, a file's name is not part of the file.**

A file is a numbered object called an **inode** (index node). The inode holds everything the system knows about the file — its size, its owner, its permissions, its timestamps, and the disk block addresses where the data actually lives. What it does *not* hold is the **name**.

The name lives in a **directory**. A directory is just an ordinary file whose contents are a simple two-column table:

Filename	Inode number
.	2341
..	118
notes.txt	5567
report	5567

Notice that `notes.txt` and `report` point at the *same* inode 5567. They are not a copy — they are two **names for one file**. That is a **hard link**. Delete one name and the file survives, because the inode keeps a **link count** and only when the count falls to zero (and no process has it open) are the data blocks freed. This is why the delete system call is literally named **unlink** — you remove a name, not a file.

What an inode contains

Stored in the inode	Stored elsewhere
File type (regular, dir, link...)	The file name → in the directory entry
Permissions (12 bits: 9 rwx + SUID/SGID/sticky)	The inode number → also in the directory entry
UID (owner) and GID (group)	The file's contents → in data blocks
File size in bytes	
Link count (number of hard links)	
Timestamps : <code>atime</code> (last access), <code>mtime</code> (last data modification), <code>ctime</code> (last inode/metadata change)	
Pointers to data blocks — typically 12 direct, 1 single-indirect, 1 double-indirect, 1 triple-indirect	
Number of blocks allocated	

There is no `ctime` = "creation time" — `ctime` is *change* time (inode changed). Classic MCQ trap. Traditional UNIX file systems store **no creation time at all** (ext4 does store `crttime`/birth time internally). This matters in forensics.

Block addressing arithmetic (a favourite numerical). With 12 direct pointers, one single-, one double- and one triple-indirect pointer, a 4 KB block size and 4-byte block addresses:

- Addresses per block = $4096 / 4 = 1024$
- Direct: $12 \times 4 \text{ KB} = 48 \text{ KB}$
- Single indirect: $1024 \times 4 \text{ KB} = 4 \text{ MB}$
- Double indirect: $1024 \times 1024 \times 4 \text{ KB} = 4 \text{ GB}$
- Triple indirect: $1024^3 \times 4 \text{ KB} = 4 \text{ TB}$
- **Maximum file size** $\approx 48 \text{ KB} + 4 \text{ MB} + 4 \text{ GB} + 4 \text{ TB} \approx 4 \text{ TB}$

Worked example — how the kernel opens `/home/ravi/notes.txt`

This is a superb 10-mark answer because it ties the whole file system together. Narrate it:

1. Path begins with `/`, so start at the **root inode**, which is **inode number 2** by convention (inode 1 is reserved for bad blocks). The kernel knows this without searching.
2. Read root's inode → get its data blocks → read them as a directory table → look up the name `home` → find inode number, say 128.
3. Check the **execute (search) permission** on `/` for this user. If absent, stop with "Permission denied".
4. Read inode 128 → confirm it is a directory → read its data blocks → look up `ravi` → get inode 4501. Check search permission again.
5. Read inode 4501 → read its data blocks → look up `notes.txt` → get inode 5567.
6. Read inode 5567 → check **read permission** on the file itself → obtain the data block pointers → read the data.
7. Allocate a **file descriptor** in the process's file-descriptor table pointing to a system file-table entry pointing to the in-core inode. Return the descriptor (first free number, typically 3, since 0/1/2 are stdin/stdout/stderr).

The takeaway sentence to write down: *directory traversal requires the execute bit on every directory in the path, and the read bit only on the final file.*

Hard links vs symbolic links — table you must be able to reproduce

	Hard link (<code>ln file link</code>)	Symbolic / soft link (<code>ln -s file link</code>)
What it is	An additional directory entry pointing to the same inode	A separate file whose data is a pathname
Inode number	Same as original	Different (its own inode)
Link count	Increments the original's count	Does not change the original's count
Across file systems	Not allowed (inode numbers are per-FS)	Allowed
To a directory	Not allowed (except <code>.</code> and <code>..</code> , made by the kernel)	Allowed
If original deleted	Link still works — data survives	Link breaks (dangling link)
Size	Same as file	Size = number of characters in the pathname
<code>ls -l</code> type char	<code>-</code> (indistinguishable from a normal file)	<code>l</code> , shown as <code>link -> target</code>

Commands: `ls -li` shows inode numbers; `ls -li` shows inode + link count; `stat file` dumps the full inode; `df -i` shows inode usage per file system; `find . -inum 5567` finds all names for an inode.

Likely exam questions

- [5] What is an inode? List the information stored in an inode.
- [5] Differentiate between hard links and soft links.

- **[5]** Why does deleting a file in UNIX not always free its disk space?
- **[10]** Explain the concept of an inode. Draw the inode structure showing direct and indirect block pointers and calculate the maximum file size for 4 KB blocks and 4-byte addresses.
- **[15]** Describe in detail how UNIX stores and locates a file. Explain directory entries, inodes, data blocks, and trace the resolution of the pathname `/home/ravi/notes.txt`.

MCQ traps

Trap	Correct answer
"The file name is stored in the inode."	False — in the directory entry.
"ctime is creation time."	False — inode change time.
"Root directory inode number?"	2 .
"Hard link across file systems?"	Not possible .
"Symbolic link to a directory?"	Possible (hard link is not).
"Which command shows inode number?"	<code>ls -li</code> or <code>stat</code> .
"Deleting the original breaks which link?"	Symbolic link (hard link still works).
"System call to delete a file?"	<code>unlink</code> .

5. Ownership and permissions — including chmod calculations

This section is the highest-yield numerical content in the file. It is explicitly named in the Computer Forensic syllabus ("Linux file system — Ownership and permissions") and implied throughout the Diploma UNIX unit. Expect both an MCQ ("`chmod 754` gives?") and a 10-mark question.

Concept

Every file has exactly one **owner** (a UID) and exactly one **group** (a GID). When you try to touch a file, the kernel classifies you into **exactly one** of three categories, and it checks that category's permissions **only**:

1. Are you the **owner**? → use the **user (u)** permissions. Stop.
2. Else, are you in the file's **group**? → use the **group (g)** permissions. Stop.
3. Else → use the **other (o)** permissions.

The word "stop" carries a mark. If you are the owner and the owner has no read permission, you **cannot read the file even if group and other can**. The checks are not cumulative and not most-permissive — they are first-match. Students get this wrong constantly.

Each category has three permission bits: **r** (read = 4), **w** (write = 2), **x** (execute = 1).

What the bits actually mean — files vs directories

The same letter means something quite different on a directory. This is the second most

common misunderstanding.

Bit	On an ordinary file	On a directory
r (4)	Read/view/copy the contents	List the names in it (<code>ls</code>) — but not their attributes without x
w (2)	Modify contents, append, truncate	Create, delete or rename entries in it — <i>regardless of the permissions on those files themselves</i>
x (1)	Execute it as a program or script	Search / traverse it — enter it with <code>cd</code> , use it as part of a pathname

Two consequences to state in an answer:

- You can **delete a file you cannot even read**, provided you have **w+x on its directory**. Deletion is a directory operation, not a file operation. This is the reason `/tmp` needs the sticky bit (below).
- r** without **x** on a directory lets you see filenames but nothing else — `ls` works, `ls -l` shows `?????`, and `cd` fails.

Reading `ls -l`

```
-rwxr-xr-- 1 ravi staff 4096 Sep  5 09:14 script.sh
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
| u  g  o  | owner group size mtime name
|          | link count
|          | file type
```

Numeric (octal) mode — the calculation

Treat each triad as a 3-bit binary number: **r = 4**, **w = 2**, **x = 1**, and add.

Octal	Binary	Symbolic	Meaning
0	000	---	no permission
1	001	--x	execute only
2	010	-w-	write only
3	011	-wx	write + execute
4	100	r--	read only
5	101	r-x	read + execute
6	110	rw-	read + write
7	111	rwx	all

Worked example 1 — symbolic → numeric

```
-rwxr-xr--
```

Triad	Bits	Arithmetic	Digit
-------	------	------------	-------

user <code>rwX</code>	111	4+2+1	7
group <code>r-X</code>	101	4+0+1	5
other <code>r--</code>	100	4+0+0	4

Answer: 754. Command: `chmod 754 script.sh`

Worked example 2 — numeric → symbolic

`chmod 640 report.txt`

Digit	Binary	Symbolic
6	110	<code>rw-</code>
4	100	<code>r--</code>
0	000	<code>---</code>

Answer: `-rw-r-----` — owner reads and writes, group reads, others nothing. This is the standard mode for a private document. (`/etc/shadow` is 640 or 600 for exactly this reason.)

Worked example 3 — the classic set

Mode	Symbolic	Typical use
777	<code>rwXrwxrwx</code>	Everything to everyone — a security hole; never use
755	<code>rwXr-xr-x</code>	Executables, scripts, and directories — owner full, everyone else read+traverse
754	<code>rwXr-xr--</code>	As above but others may only read
700	<code>rwX-----</code>	Private directory (e.g. <code>~/.ssh</code>)
666	<code>rw-rw-rw-</code>	Everyone can edit a data file; no execute
644	<code>rw-r--r--</code>	Default for ordinary files — owner writes, world reads
640	<code>rw-r--r--</code>	Group-private document
600	<code>rw-----</code>	Private file (e.g. <code>~/.ssh/id_rsa</code> ; SSH <i>refuses</i> to use a key that is more open)
444	<code>r--r--r--</code>	Read-only for all
000	<code>-----</code>	No access (root still bypasses everything)

Worked example 4 — symbolic chmod (relative changes)

Syntax: `chmod [who][operator][permission] file`

- **who:** `u` user, `g` group, `o` other, `a` all (default if omitted)
- **operator:** `+` add, `-` remove, `=` set exactly (clears the rest)

Start with `report.txt` at `-rw-r--r--` (644).

Command	Effect	Resulting mode
<code>chmod u+x report.txt</code>	add execute for owner	<code>-rwxr--r--</code> = 744
<code>chmod go-r report.txt</code>	remove read from group and other	<code>-rwx-----</code> = 700
<code>chmod a+r report.txt</code>	add read for all	<code>-rwxr--r--</code> = 744
<code>chmod g=rw report.txt</code>	set group to exactly rw (x cleared)	<code>-rwxrw-r--</code> = 764
<code>chmod o= report.txt</code>	set other to nothing	<code>-rwxrw----</code> = 760
<code>chmod 755 report.txt</code>	absolute set	<code>-rwxr-xr-x</code> = 755

Note the difference between `+/-` (relative, leave other bits alone) and `=` (absolute for that category). `chmod -R 755 dir` applies recursively.

Worked example 5 — umask

New files are **not** created with 777. The shell's **umask** is a mask of bits to *remove*.

- Default **base** permission: **666** for ordinary files, **777** for directories.
(Files never get execute by default — a deliberate safety decision.)
- Effective permission = **base AND NOT umask**, which for exam purposes you compute as **base – umask** (digit-wise subtraction, never going below 0).

With the common default **umask 022**:

	File	Directory
Base	666	777
umask	022	022
Result	644 (<code>rw-r--r--</code>)	755 (<code>rwxr-xr-x</code>)

With **umask 027**:

	File	Directory
Base	666	777
umask	027	027
Result	640 (<code>rw-r-----</code>)	750 (<code>rwxr-x---</code>)

With **umask 077** (maximum privacy): files **600**, directories **700**.

Careful with the subtraction shortcut. It is really a bitwise AND-NOT. Take base 666 and umask 033: naive subtraction gives 633, but the true answer is 666 AND NOT 033:

6 = 110, mask 3 = 011, NOT 011 = 100, 110 AND 100 = 100 = 4. So the answer is **644**, not 633. Rule: *the mask can only remove bits that are present*. Since 6 (`rw-`) has no execute bit, masking out execute changes nothing. Subtraction only works when no digit would go negative and no absent bit is being removed — with umask 022 and 027 it is safe, with odd

digits against 6 it is not. **Always do the binary version if any digit is odd.**

Set it with `umask 027`; display it with `umask` (or `umask -S` for symbolic).

Worked example 6 — special permission bits

Beyond the nine bits there are three more, giving a **four-digit** octal mode `chmod 4755`.

Bit	Octal	Symbol in <code>ls -l</code>	Meaning
SUID (Set User ID)	4000	s in the user execute position (<code>-rwsr-xr-x</code>)	While running, the process takes the owner's effective UID, not the caller's
SGID (Set Group ID)	2000	s in the group execute position (<code>-rwxr-sr-x</code>)	On a file: run with the file's group. On a directory: new files inherit the directory's group — used for shared project directories
Sticky bit	1000	t in the other execute position (<code>drwxrwxrwt</code>)	On a directory: only the file's owner (or root) may delete or rename entries in it, even though everyone can write. This is what makes world-writable <code>/tmp</code> safe

Worked: `chmod 4755 /usr/bin/passwd` → `-rwsr-xr-x`. Why does `passwd` need it? Because changing your password means writing to `/etc/shadow`, which is owned by root and mode 640. An ordinary user cannot write it. SUID-root on `/usr/bin/passwd` means the *program* runs as root for its lifetime, does the one controlled edit, and exits. **SUID binaries are therefore the number-one privilege-escalation target** — a Forensic-paper point. Find them with:

```
find / -perm -4000 -type f 2>/dev/null      # all SUID files
find / -perm -2000 -type f 2>/dev/null      # all SGID files
```

If the corresponding execute bit is **absent**, the letter appears as a **capital S** or **T** — a classic MCQ. `-rwSr--r--` means SUID set but owner has no execute (meaningless, and a red flag).

Ownership commands

Command	Purpose	Example
<code>chown</code>	Change owner (root only, in practice)	<code>chown ravi file.txt</code>
<code>chown user:group</code>	Change owner and group at once	<code>chown ravi:staff file.txt</code>
<code>chown -R</code>	Recursively	<code>chown -R ravi:staff /project</code>
<code>chgrp</code>	Change group only	<code>chgrp staff file.txt</code>
<code>chmod</code>	Change permissions	<code>chmod 755 file.txt</code>
<code>umask</code>	Set default permission mask	<code>umask 022</code>
<code>id</code>	Show your UID, GID, groups	<code>id ravi</code>
<code>groups</code>	Show group memberships	<code>groups ravi</code>

<code>su</code>	Switch user	<code>su - ravi</code>
<code>sudo</code>	Run a single command as another user (config <code>/etc/sudoers</code> , edit via <code>visudo</code>)	<code>sudo apt update</code>
<code>passwd</code>	Change password	<code>passwd ravi</code>
<code>useradd</code> / <code>userdel</code> / <code>usermod</code>	Manage accounts	<code>useradd -m -s /bin/bash ravi</code>

Only the owner of a file or root can change its permissions — even a user with write permission cannot. And **root bypasses all permission checks** (except, on most systems, the execute bit if nobody has it).

Likely exam questions

- **[5]** Explain the UNIX file permission system with reference to user, group and other.
- **[5]** What is the significance of `chmod 754`? Show the calculation.
- **[5]** What is umask? If umask is 027, what are the default permissions of a newly created file and directory? Show the working.
- **[5]** Explain SUID, SGID and the sticky bit with one example each.
- **[10]** Describe file ownership and permissions in UNIX. Explain both symbolic and numeric modes of `chmod` with at least four worked examples, and explain how permissions differ in meaning for files and directories.
- **[15]** Explain the security model of the UNIX file system: ownership, the three permission categories and the first-match rule, the nine permission bits, the three special bits, umask, and how SUID programs such as `passwd` work and why they are a security risk.

MCQ traps

Trap	Correct answer
" <code>chmod 754</code> ?"	<code>rwxr-xr--</code> .
" <code>rw-r-----</code> in octal?"	640 .
"Default umask 022 → new <i>file</i> permission?"	644 (not 755 — files have no default execute).
"Default umask 022 → new <i>directory</i> permission?"	755 .
"Owner is denied but group allowed — can the owner read?"	No . First-match, checks stop at owner.
"Can you delete a read-only file?"	Yes , if you have <code>w+x</code> on the directory.
" <code>t</code> at the end of <code>drwxrwxrwt</code> ?"	Sticky bit (<code>/tmp</code>).
"SUID octal value?"	4000 . SGID 2000 , sticky 1000 .
"Which permission lets you <code>cd</code> into a directory?"	execute (x) , not read.
"Who can change a file's permissions?"	The owner or root only.
"Value of <code>x</code> ?"	4 . (<code>w</code> =2, <code>x</code> =1.)

"Capital **s** in permission string?"

SUID set **without** the underlying execute bit.

6. Booting, login, startup and shutdown

Concept

From power-on to a login prompt, control is handed along a chain, and each link only knows enough to find and start the next one. The chain gets shorter every step in terms of hardware knowledge and longer in terms of capability.

The booting procedure — six stages (learn the names in order)

#	Stage	What happens
1	BIOS / UEFI	Firmware in ROM wakes up. Runs POST (Power-On Self Test) — checks CPU, RAM, keyboard, devices. Reads the boot-device order from CMOS. Loads the first sector of the chosen device into memory.
2	MBR / boot sector	The Master Boot Record is the first 512 bytes of the disk: 446 bytes of bootstrap code + 64 bytes partition table (4 entries × 16 B) + 2-byte magic number 0x55AA . It locates and loads the boot loader. (GPT/UEFI systems use an EFI System Partition instead.)
3	Boot loader	GRUB / GRUB2 (or LILO historically, or boot on classic UNIX). Presents the kernel menu, loads the kernel image /boot/vmlinuz and the initrd / initramfs (a temporary root file system holding the drivers needed to mount the real root) into memory. Passes kernel parameters.
4	Kernel initialisation	Kernel decompresses itself, initialises memory management and the scheduler, detects and initialises hardware, loads drivers, mounts the root file system (read-only first, then read-write), and then creates the very first process.
5	init / systemd	The kernel starts /sbin/init — PID 1 , the ancestor of every other process. Classic SysV init reads /etc/inittab , determines the default runlevel , and runs the scripts in /etc/rc.d/ rcN.d/ . Modern Linux uses systemd (targets instead of runlevels) or Upstart.
6	Getty & login	init spawns a getty on each terminal, which prints the login: prompt and hands over to /bin/login .

Draw this as a **vertical flowchart of six boxes with downward arrows**, annotating box 4 with "kernel loaded into memory, stays resident" and box 5 with "PID 1".

Runlevels (SysV) — a table examiners love

Runlevel	Meaning
0	Halt / shutdown
1 or s/s	Single-user mode — maintenance, root only, no networking
2	Multi-user without networking/NFS (Debian: full multi-user with GUI)
3	Full multi-user, text/console mode with networking

4	Unused / user-definable
5	Full multi-user with GUI (X11 / display manager)
6	Reboot

Commands: `runlevel` (shows previous and current), `init 3` / `telinit 3` to change, `who -r`. systemd equivalents: `multi-user.target` (≈ 3), `graphical.target` (≈ 5), `rescue.target` (≈ 1); manage with `systemctl`.

The login process

- `getty` opens a terminal device and displays `login:`.
- User types a username; `getty` execs `/bin/login`, which prompts for `Password:` with **echo disabled**.
- `login` looks up the user in `/etc/passwd`, and the password hash in `/etc/shadow`.
- It hashes the typed password with the **salt** stored alongside the hash and compares. It never decrypts anything — the stored value is a **one-way hash** (crypt/MD5/SHA-512/yescrypt).
- On failure: generic "Login incorrect" (deliberately not saying which field was wrong), logged to `/var/log/btmp` and `auth.log`. Three failures typically drop the connection.
- On success it:
 - sets UID and GID from fields 3 and 4 of `/etc/passwd`,
 - changes directory to the **home directory** (field 6),
 - executes the **login shell** (field 7),
 - records the session in `/var/run/utmp` and `/var/log/wtmp`,
 - prints `/etc/motd` (message of the day) and mail notification.
- The shell then runs the **initialisation files** in order:

Scope	Bourne/bash login shell	Read when
System-wide	<code>/etc/profile</code>	Every login
User	<code>~/.bash_profile</code> → else <code>~/.bash_login</code> → else <code>~/.profile</code>	Login shells only
User (non-login/interactive)	<code>~/.bashrc</code>	Every new interactive shell
Logout	<code>~/.bash_logout</code>	On exit

(csh uses `.login`, `.cshrc`, `.logout`.) Forensically, `~/.bashrc` and `~/.bash_profile` are common persistence locations for attackers.

The `/etc/passwd` record — seven colon-separated fields

<pre>ravi : x : 1001 : 1001 : Ravi Kumar,,, : /home/ravi : /bin/bash</pre>						
1	2	3	4	5	6	7
#	Field	Note				

1	Username	
2	Password placeholder	<code>x</code> means the hash is in <code>/etc/shadow</code> . An empty field means no password — a red flag
3	UID	0 = root . 1–999 system accounts; 1000+ ordinary users (500+ on older RHEL)
4	GID	Primary group
5	GECOS / comment	Full name, office, phone
6	Home directory	
7	Login shell	<code>/sbin/nologin</code> or <code>/bin/false</code> = account cannot log in

Any account with UID 0 is root, whatever it is called — the single most important line to grep for in a Linux forensic examination.

Startup and shutdown commands

Command	Effect
<code>shutdown -h now</code>	Halt immediately
<code>shutdown -h +10 "message"</code>	Halt in 10 minutes, broadcasting a warning to all users
<code>shutdown -r now</code>	Reboot immediately
<code>shutdown -c</code>	Cancel a pending shutdown
<code>halt</code>	Stop the CPU (may not power off)
<code>poweroff</code>	Halt and cut power
<code>reboot</code>	Restart
<code>init 0</code> / <code>init 6</code>	Shutdown / reboot via runlevel
<code>sync</code>	Flush buffered disk writes to disk — traditionally typed before halting
<code>systemctl poweroff</code> / <code>reboot</code> / <code>rescue</code>	systemd equivalents

Why `shutdown` and not just pulling the plug: UNIX **buffers** disk writes in memory. An orderly shutdown warns users, kills processes with **SIGTERM** then **SIGKILL**, unmounts file systems, and syncs the buffer cache. Yanking power leaves the file system **dirty**, forcing a `fsck` at next boot and risking data loss. Say this in a 5-mark answer.

Likely exam questions

- **[5]** Explain the stages of the UNIX booting procedure.
- **[5]** Describe the login process in UNIX.
- **[5]** Explain the fields of the `/etc/passwd` file.
- **[5]** What are runlevels? Tabulate runlevels 0–6.
- **[5]** Why must a UNIX system be shut down properly? List the shutdown commands.

- **[10]** With a neat flowchart, explain the complete boot sequence of a UNIX/Linux system from power-on to the shell prompt, including the role of BIOS, MBR, GRUB, the kernel, init and getty.
- **[15]** Explain booting, the login process, password verification and shell initialisation in UNIX, including the roles of `/etc/passwd`, `/etc/shadow`, `/etc/profile` and `~/.bashrc`.

MCQ traps

Trap	Correct answer
"First process created by the kernel?"	<code>init</code> (or <code>systemd</code>), PID 1 .
"PID 0?"	The scheduler / swapper — not init.
"Which runlevel is single-user?"	1 . Halt is 0 , reboot is 6 .
"Size of MBR?"	512 bytes .
"Which prints the login prompt?"	<code>getty</code> .
"Where are password hashes?"	<code>/etc/shadow</code> .
"UID of root?"	0 .
"POST is done by?"	BIOS/UEFI firmware .
"What does <code>sync</code> do?"	Flushes buffers to disk.
" <code>init 6</code> does?"	Reboot (students often say shutdown).

7. Essential commands — organised by purpose

This is the MCQ engine room. Revise the tables; do not try to memorise every option. Learn each command's *one-line purpose* first, then the two or three options that get asked.

7.1 Navigation and directory management

Command	Purpose	Key options / examples
<code>pwd</code>	P rint w orking d irectory	<code>pwd</code> → <code>/home/ravi</code>
<code>cd</code>	Change directory	<code>cd /etc</code> ; <code>cd ..</code> (parent); <code>cd</code> or <code>cd ~</code> (home); <code>cd -</code> (previous directory); <code>cd /</code> (root)
<code>ls</code>	List directory contents	<code>-l</code> long listing; <code>-a</code> all incl. hidden; <code>-A</code> hidden but not <code>./</code> <code>..</code> ; <code>-i</code> inode; <code>-R</code> recursive; <code>-t</code> sort by mtime; <code>-r</code> reverse; <code>-S</code> sort by size; <code>-h</code> human-readable sizes; <code>-d</code> the directory itself not contents; <code>-F</code> append type indicator (<code>/</code> dir, <code>*</code> exec, <code>@</code> link)
<code>mkdir</code>	Make directory	<code>mkdir docs</code> ; <code>mkdir -p a/b/c</code> (create parents as needed); <code>mkdir -m 755 dir</code>

<code>rmdir</code>	Remove directory — only if empty	<code>rmdir docs; rmdir -p a/b/c</code>
<code>tree</code>	Show directory tree graphically	<code>tree -L 2</code>

`md` appears in the Diploma syllabus — it is the DOS name and, on UNIX, only exists if someone has aliased it; the real command is `mkdir`. Mention both in an answer.

Absolute vs relative paths — worth a 2-mark definition: an **absolute** path starts at `/` and is unambiguous (`/home/ravi/notes.txt`); a **relative** path starts from the current directory (`docs/notes.txt`, `../ravi/notes.txt`). Shorthands: `.` current, `..` parent, `~` your home, `~ravi` ravi's home, `-` previous directory.

7.2 File creation, display and management

Command	Purpose	Key options / examples
<code>cat</code>	C atenate and display files; also create small files	<code>cat f1</code> display; <code>cat f1 f2 > f3</code> concatenate ; <code>cat > new</code> create (end with Ctrl+D); <code>cat >> f</code> append; <code>-n</code> number all lines; <code>-b</code> number non-blank lines; <code>-A/-v</code> show non-printing chars; <code>-s</code> squeeze blank lines
<code>touch</code>	Create an empty file / update timestamps	<code>touch f; -t 202609051200 f</code> set specific time (note: anti-forensic timestamping)
<code>cp</code>	Copy	<code>cp src dst; cp f1 f2 dir/; -r/-R</code> recursive (required for directories); <code>-i</code> interactive prompt; <code>-p</code> preserve mode/owner/timestamps; <code>-a</code> archive (= <code>-dpR</code>); <code>-u</code> only if newer; <code>-v</code> verbose
<code>mv</code>	M ove or r ename (there is no separate <code>rename</code>)	<code>mv old new</code> rename; <code>mv f dir/</code> move; <code>-i</code> prompt; <code>-f</code> force; <code>-n</code> no-clobber
<code>rm</code>	Remove file	<code>rm f; -i</code> confirm; <code>-r</code> recursive (directories); <code>-f</code> force, no prompt, ignore missing. <code>rm -rf /</code> is catastrophic
<code>ln</code>	Create link	<code>ln f hard; ln -s f soft</code>
<code>file</code>	Identify file type by magic number	<code>file report</code> → ASCII text
<code>stat</code>	Display full inode information	<code>stat f</code>
<code>du</code>	D isk u usage by file/directory	<code>du -sh dir</code> (summary, human-readable)
<code>df</code>	D isk f ree by file system	<code>df -h; df -i</code> (inodes)
<code>find</code>	Search the hierarchy by attribute	<code>find /home -name "*.log"; -type f/d/l; -size +10M; -mtime -7</code> (modified in last 7 days); <code>-user ravi; -perm -4000; -exec rm {} \;</code>
<code>locate</code>	Fast filename search via a prebuilt database	<code>locate passwd</code> (update with <code>updatedb</code>)

<code>which / whereis</code>	Locate an executable in <code>PATH</code> / locate binary+source+man	<code>which ls; whereis ls</code>
<code>tar</code>	Archive	<code>tar -cvf a.tar dir</code> create; <code>-xvf</code> extract; <code>-tvf</code> list; <code>-z</code> gzip, <code>-j</code> bzip2
<code>gzip/gunzip, zip/unzip, bzip2</code>	Compression	<code>gzip f</code> → <code>f.gz</code>

7.3 File processing / filter commands — the big Diploma list

A **filter** is a command that reads from **standard input**, transforms, and writes to **standard output** — so it can sit in the middle of a pipeline. `grep`, `sort`, `wc`, `cut`, `tr`, `head`, `tail`, `uniq`, `sed`, `awk`, `more`, `less`, `tee` are filters. `ls`, `cp`, `mv`, `rm`, `mkdir` are **not** filters (they don't read stdin).

Command	Purpose	Key options / examples
<code>wc</code>	Word count — prints lines, words, characters (in that order)	<code>wc f</code> → 12 84 512 <code>f</code> ; <code>-l</code> lines only; <code>-w</code> words; <code>-c</code> bytes; <code>-m</code> characters; <code>-L</code> longest line
<code>head</code>	First part of a file — default 10 lines	<code>head f</code> ; <code>head -5 f</code> or <code>head -n 5 f</code> ; <code>-c 100</code> first 100 bytes
<code>tail</code>	Last part — default 10 lines	<code>tail f</code> ; <code>tail -20 f</code> ; <code>tail -f logfile</code> follow a growing file (log monitoring); <code>tail +5 f</code> from line 5 onward
<code>cut</code>	Extract columns / fields from each line	<code>cut -c1-10 f</code> characters 1–10; <code>cut -d: -f1,7 /etc/passwd</code> fields 1 and 7 with <code>:</code> delimiter; <code>-d</code> delimiter (default TAB), <code>-f</code> field list
<code>paste</code>	Merge lines of files side by side (the horizontal opposite of <code>cat</code>)	<code>paste f1 f2</code> ; <code>-d', '</code> delimiter; <code>-s</code> serial (one file per line)
<code>join</code>	Relational join of two sorted files on a common field	<code>join f1 f2</code> ; <code>-1 2 -2 1</code> join field 2 of file1 with field 1 of file2; <code>-t:</code> delimiter. Both files must be sorted on the join field
<code>split</code>	Split a file into fixed-size pieces	<code>split -l 100 big.txt part_</code> → <code>part_aa, part_ab...</code> ; <code>-b 1M</code> by size
<code>sort</code>	Sort lines	<code>sort f</code> ; <code>-r</code> reverse; <code>-n</code> numeric (else "10" sorts before "9"); <code>-k2</code> by field 2; <code>-t:</code> field separator; <code>-u</code> unique; <code>-f</code> fold case; <code>-M</code> month; <code>-o out</code> write to file; <code>-c</code> check if sorted
<code>grep</code>	G lobal R egular E xpression P rint — search lines matching a pattern	<code>grep "error" f</code> ; <code>-i</code> ignore case; <code>-v</code> invert (lines NOT matching); <code>-n</code> line numbers; <code>-c</code> count only; <code>-l</code> list filenames only; <code>-w</code> whole word; <code>-r/-R</code> recursive; <code>-A3 -B3 -C3</code> context lines; <code>-E</code> extended regex (= <code>egrep</code>); <code>-F</code> fixed strings (= <code>fgrep</code>)

egrep	Extended grep — supports <code>+ ? \ () { }</code> without backslashes	<code>egrep "cat\ dog" f</code> ; <code>egrep "[0-9]{3}" f</code> . Equivalent to <code>grep -E</code>
fgrep	Fixed-string grep, no regex — fastest	<code>fgrep "a.b" f</code> matches literal <code>a.b</code>
tr	T ranslate or delete characters. Reads only from stdin	<code>tr 'a-z' 'A-Z' < f</code> lowercase→uppercase; <code>tr -d ' '</code> delete spaces; <code>tr -s ' '</code> squeeze repeats; <code>tr -c</code> complement
comm	C ommon — compare two sorted files, three columns: lines only in file1, only in file2, in both	<code>comm f1 f2</code> ; <code>comm -12 f1 f2</code> show only common lines; <code>-1</code> suppress col 1, <code>-2</code> col 2, <code>-3</code> col 3
cmp	Byte-by-byte comparison; reports the first difference (byte and line number). Works on binary files	<code>cmp f1 f2</code> → <code>f1 f2 differ: byte 17, line 2</code> ; <code>-s</code> silent (exit status only)
diff	Line-by-line difference; reports what to change to make file1 into file2	<code>diff f1 f2</code> ; <code>-u</code> unified format; <code>-i</code> ignore case; <code>-y</code> side-by-side; <code>-r</code> recursive. Output codes a append, d delete, c change
more	Page through output, forward only (older)	<code>more f</code> ; SPACE next page, ENTER next line, <code>q</code> quit, <code>/pat</code> search
less	Page through output, forward and backward , doesn't load whole file — "less is more"	<code>less f</code> ; arrows, <code>b</code> back, <code>G</code> end, <code>g</code> start, <code>/pat</code> , <code>q</code>
uniq	Remove adjacent duplicate lines — must <code>sort</code> first	<code>sort f \ uniq</code> ; <code>-c</code> prefix counts; <code>-d</code> only duplicates; <code>-u</code> only unique
nl	Number lines	<code>nl f</code>
tee	Write stdin to a file and to stdout	<code>ls \ tee out.txt \ wc -l</code> ; <code>-a</code> append
sed	S tream e ditor — non-interactive editing	<code>sed 's/old/new/g' f</code> ; <code>sed -n '5,10p' f</code> ; <code>sed '/^\$/d' f</code> delete blank lines; <code>-i</code> edit in place
awk	Pattern-scanning and field-processing language	<code>awk '{print \$1,\$3}' f</code> ; <code>awk -F: '\$3>1000 {print \$1}' /etc/passwd</code> ; <code>awk 'END{print NR}' f</code>

Worked pipeline examples (excellent 5-mark answers)

Task	Command
Count the number of users on the system	<code>wc -l < /etc/passwd</code>
List all login shells in use, no duplicates	<code>cut -d: -f7 /etc/passwd \ sort \ uniq</code>

Find all users with UID 0 (root equivalents)	<code>awk -F: '\$3==0 {print \$1}' /etc/passwd</code>
Top 5 largest files in a directory	<code>ls -lS \ head -6</code>
Count how many times "error" appears in a log	<code>grep -c "error" /var/log/syslog</code>
Find the 10 most frequent words in a file	<code>tr -cs 'A-Za-z' '\n' < f \ sort \ uniq -c \ sort -rn \ head -10</code>
Display lines 10 to 20 of a file	<code>head -20 f \ tail -11</code> or <code>sed -n '10,20p' f</code>
List currently logged-in users, sorted, unique	<code>who \ cut -d' ' -f1 \ sort -u</code>
Total disk usage of *.log files	<code>find . -name "*.log" -exec du -ch {} + \ tail -1</code>

cmp vs diff vs comm — a guaranteed question:

	cmp	diff	comm
Compares	Byte by byte	Line by line	Line by line
Files must be sorted	No	No	Yes
Reports	First difference (byte, line)	All differences + how to fix	Three columns: unique-to-1, unique-to-2, common
Binary files	Yes	Poorly ("Binary files differ")	No

7.4 File formatting and printing

Command	Purpose	Key options
pr	Format a file for printing : paginate and add a header (date, filename, page number) and 5-line header + 5-line trailer margins	see the table below
lp	Send a file to the printer (System V) — returns a request id	<code>lp -d laser f; -n 3 copies; -t "title"</code>
<code>lpr</code>	BSD equivalent	<code>lpr -P laser f</code>
<code>lpstat</code> / <code>lpq</code>	Show printer/queue status	<code>lpstat -t, lpq</code>
<code>cancel</code> / <code>lprm</code>	Cancel a print job	<code>cancel laser-42</code>
<code>fmt</code>	Simple paragraph reformatter	<code>fmt -w 60 f</code>

<code>nl</code>	Number lines before printing	
-----------------	------------------------------	--

pr options — the Diploma syllabus says "**pr** with options", so know these:

Option	Effect
<code>-h "text"</code>	Use <i>text</i> as the header in place of the filename
<code>-l n</code>	Set page length to <i>n</i> lines (default 66)
<code>-w n</code>	Set page width to <i>n</i> columns (default 72)
<code>-n</code>	Number the lines
<code>-d</code>	Double-space the output
<code>-t</code>	Suppress the header and trailer entirely
<code>-o n</code>	Offset each line by <i>n</i> spaces (left margin)
<code>-k</code>	Produce output in <i>k</i> columns (e.g. <code>pr -3 f</code>)
<code>-m</code>	Merge files, printing them side by side in parallel columns
<code>+n</code>	Begin printing at page n
<code>-a</code>	With <code>-k</code> , fill columns across rather than down

Example: `pr -h "Salary Report" -l 60 -n -d salary.txt | lp -d laser`

— paginate `salary.txt` with a custom header, 60-line pages, numbered and double-spaced, and send the result to the laser printer.

7.5 Online help

Command	Purpose	Example
<code>man</code>	Display the man ual page for a command	<code>man ls</code> ; <code>man 5 passwd</code> (section 5); <code>man -k copy</code> keyword search; <code>man man</code>
<code>help</code>	Shell built-in help — works only for built-ins like <code>cd</code> , <code>echo</code> , <code>export</code>	<code>help cd</code> ; <code>help</code> alone lists all built-ins
<code>info</code>	GNU hypertext documentation, usually fuller than <code>man</code>	<code>info coreutils</code>
<code>whatis</code>	One-line description from the man database	<code>whatis ls</code>
<code>apropos</code>	Search man page descriptions by keyword (= <code>man -k</code>)	<code>apropos network</code>
<code>--help</code>	Most GNU commands' own brief usage	<code>ls --help</code>
<code>type</code>	Tell whether a name is a built-in, alias, or file	<code>type cd</code> → "cd is a shell builtin"

Manual sections — an MCQ favourite, because `man passwd` gives the *command* while `man 5 passwd` gives the *file format*:

Section	Contents
1	User commands (<code>ls</code> , <code>cp</code>)
2	System calls (<code>open</code> , <code>fork</code>)
3	Library functions (<code>printf</code> , <code>malloc</code>)
4	Special files / devices (<code>/dev/null</code>)
5	File formats and conventions (<code>/etc/passwd</code> , <code>fstab</code>)
6	Games
7	Miscellaneous / conventions
8	System administration commands (<code>mount</code> , <code>shutdown</code>)
9	Kernel routines

Man page structure: NAME, SYNOPSIS, DESCRIPTION, OPTIONS, EXAMPLES, FILES, SEE ALSO, BUGS, AUTHOR. Navigation inside `man` is `less`: SPACE, `b`, `/pattern`, `n`, `q`.

7.6 Mathematical commands

Command	Purpose	Worked examples
<code>bc</code>	B asic c alculator — an arbitrary-precision interactive calculator language. Integer arithmetic by default ; set <code>scale</code> for decimals	<pre>echo "5+3" \ bc → 8 echo "10/3" \ bc → 3 (integer!) echo "scale=4; 10/3" \ bc → 3.3333 echo "scale=2; sqrt(2)" \ bc -l → 1.41 echo "obase=2; 25" \ bc → 11001 (decimal→binary) echo "ibase=2; 11001" \ bc → 25 echo "2^10" \ bc → 1024 bc -l loads the math library (s(), c(), a(), l(), e())</pre>

expr	Evaluate a simple expression — integer only , arguments must be space-separated	<pre>expr 5 + 3 → 8 expr 5 * 3 → 15 (* must be escaped — the shell would glob it) expr 10 / 3 → 3 expr 10 % 3 → 1 x=\$(expr \$x + 1) classic increment in old shell scripts expr length "hello" → 5 expr substr "hello" 2 3 → ell expr index "hello" 1 → 3</pre>
factor	Print the prime factorisation of an integer	<pre>factor 60 → 60: 2 2 3 5 factor 97 → 97: 97 (prime) factor 100 → 100: 2 2 5 5</pre>
units	Interactive unit conversion	<pre>units then You have: 10 km / You want: miles → * 6.2137119 Non-interactive: units "5 kg" "pounds" → * 11.023113</pre>
dc	Reverse-Polish desk calculator	<pre>echo "5 3 + p" \ dc → 8</pre>
seq	Generate a number sequence	<pre>seq 1 5 → 1 2 3 4 5; seq 0 2 10 step 2</pre>
\$(())	Shell arithmetic expansion — the modern replacement for expr	<pre>echo \$((5+3)) → 8; x=\$((x+1))</pre>

Note for MCQs: the trap is that both **bc** (without **scale**) and **expr** do **integer** division, so **10/3** is **3**, not **3.33**. And in **expr**, **expr 5+3** (no spaces) prints the *string* **5+3**, not **8**.

7.7 Communication commands

Command	Purpose	Example / notes
write	Send a message to one logged-in user's terminal, line by line, in real time. Terminate with Ctrl+D	<pre>write ravi then type; write ravi pts/2 to a specific terminal. Fails if the recipient has mesg n</pre>
wall	Write to all logged-in users — a broadcast. Usually reserved for the superuser (shutdown warnings)	<pre>wall "System going down at 5 pm"; wall < message.txt</pre>

<code>mail / mailx</code>	Send and read electronic mail ; store-and-forward, so the recipient need not be logged in	Send: <code>mail -s "Subject" ravi < body.txt</code> , or <code>mail ravi</code> then type and end with <code>.</code> on a line by itself or Ctrl+D. Read: <code>mail</code> then <code>n</code> next, <code>d</code> delete, <code>s</code> file save, <code>r</code> reply, <code>q</code> quit. Mailbox: <code>/var/spool/mail/<user></code> or <code>/var/mail/<user></code>
<code>mesg</code>	Control whether others may <code>write</code> to your terminal	<code>mesg n deny</code> , <code>mesg y</code> allow, <code>mesg</code> show current state
<code>talk</code>	Two-way interactive split-screen chat (vs <code>write</code> , which is one-way)	<code>talk ravi@host</code>
<code>finger</code>	Display information about a user (login name, real name, idle time, last login)	<code>finger ravi</code>
<code>news</code>	Read system news items	<code>news -a</code>
<code>ssh / scp / sftp</code>	Secure remote login and file transfer (replaced insecure <code>telnet</code> , <code>rlogin</code> , <code>ftp</code>)	<code>ssh ravi@host</code> ; <code>scp f ravi@host:/tmp</code>

write vs mail — a standard 5-mark comparison:

	<code>write</code>	<code>mail</code>
Recipient must be logged in	Yes	No
Delivery	Immediate, to the terminal	Stored in a mailbox, read later
Direction	One-way (both parties can run it for a conversation)	One-way, asynchronous
Scope	One user	One or many, local or remote
Blocked by	<code>mesg n</code>	Nothing

7.8 System information and status

Command	Purpose	Output / options
<code>who</code>	List all users currently logged in — name, terminal, login time, host	<code>who</code> → <code>ravi pts/0 Sep 5 09:14 (192.168.1.5)</code> ; <code>who -H</code> headings; <code>who -q</code> count only; <code>who -r</code> runlevel; <code>who -b</code> last boot time
<code>who am i</code>	Show just your own login line (i.e. <code>who</code> filtered to your terminal). Equivalent to <code>who -m</code> . Shows the login/real user	<code>who am i</code> → <code>ravi pts/0 Sep 5 09:14</code>
<code>whoami</code>	Print only the effective username — one word, no other fields	<code>whoami</code> → <code>ravi</code>

w	Who is logged in and what they are running , plus load average and idle time	w
users	Space-separated list of logged-in usernames	users
tty	Print the filename of the terminal connected to standard input	tty → /dev/pts/0; tty -s silent (script test)
date	Display or set the system date and time	date → Sat Sep 5 09:14:03 IST 2026 date +"%d-%m-%Y" → 05-09-2026 date +"%H:%M:%S"; date +%s epoch seconds; date -s "... " set (root only) Formats: %d day, %m month, %Y 4-digit year, %y 2-digit, %H hour, %M minute, %S second, %A weekday name, %B month name, %j day of year
cal	Display a calendar	cal current month; cal 2026 whole year; cal 9 2026 September 2026 (month first, then year); cal -3 prev/current/next; cal -y
uname	System information	uname -a all; -s kernel name; -r kernel release ; -n hostname; -m machine hardware
hostname	Show/set the system name	
uptime	How long the system has been up + load averages (1, 5, 15 min)	
id	Current UID, GID and groups	id → uid=1001(ravi) gid=1001(ravi) groups=...
logname	The name the user logged in with	
last	Login history (from /var/log/wtmp) — forensic gold	last, last ravi, last -x (shutdowns/runlevels)
free	Memory usage	free -h
df / du	Disk free / disk usage	df -h, du -sh *
history	Previously typed commands	history 20, !45 re-run, !! last command
echo	Display a line of text or a variable	echo "Hello", echo \$HOME, echo -n no newline, echo -e interpret escapes
clear	Clear the terminal screen	
exit / logout / Ctrl+D	End the session	

who VS **who am i** VS **whoami** is asked almost every year:

- **who** → **everyone** logged in.
- **who am i** → **one line** about you, in **who** format (terminal, time, host).
- **whoami** → **one word**, your effective username. After **su - root**, **whoami** prints **root** but **who am i** still prints your **original login name** — the difference between effective and real identity, and a genuinely useful forensic distinction.

7.9 The vi editor (three modes — a standard 5-marker)

Mode	Entered by	Purpose
Command mode	Default on opening; Esc from anywhere	Navigation and manipulation — keystrokes are commands, not text
Insert / Input mode	i insert before cursor, a append after, o open line below, O above, I start of line, A end of line	Typing text
Ex / Last-line / Escape mode	: from command mode	File-level commands: :w save, :q quit, :wq or ZZ save+quit, :q! quit discarding, :set nu line numbers, :%s/old/new/g global substitute, :1,10d delete lines

Command-mode essentials: **h j k l** (left/down/up/right), **x** delete char, **dd** delete line, **3dd** delete 3 lines, **yy** yank/copy line, **p** paste, **u** undo, **.** repeat, **G** last line, **1G/gg** first line, **/pat** search forward, **n** next match.

7.10 Master quick-reference table (last-night revision)

Command	One-line purpose
<code>pwd</code>	Print working directory
<code>cd</code>	Change directory
<code>ls</code>	List directory contents
<code>mkdir</code>	Create directory
<code>rmdir</code>	Remove empty directory
<code>cat</code>	Display / create / concatenate files
<code>touch</code>	Create empty file, update timestamp
<code>cp</code>	Copy files/directories
<code>mv</code>	Move or rename
<code>rm</code>	Delete files (-r for directories)
<code>ln</code>	Create hard/symbolic link

<code>file</code>	Identify file type
<code>find</code>	Search hierarchy by attribute
<code>wc</code>	Count lines, words, characters
<code>head</code>	First 10 lines
<code>tail</code>	Last 10 lines (<code>-f</code> follow)
<code>cut</code>	Extract columns/fields
<code>paste</code>	Merge files side by side
<code>join</code>	Join two sorted files on a common field
<code>split</code>	Split a file into pieces
<code>sort</code>	Sort lines
<code>uniq</code>	Remove adjacent duplicates
<code>grep</code>	Search for a pattern
<code>egrep</code>	Extended-regex grep (<code>grep -E</code>)
<code>fgrep</code>	Fixed-string grep
<code>tr</code>	Translate/delete characters (stdin only)
<code>comm</code>	Compare two sorted files, 3 columns
<code>cmp</code>	Byte comparison, first difference
<code>diff</code>	Line differences and how to fix
<code>more</code>	Page forward
<code>less</code>	Page both ways
<code>tee</code>	Split output to file and screen
<code>sed</code>	Stream editor
<code>awk</code>	Field/pattern processing language
<code>pr</code>	Paginate/format for printing
<code>lp</code>	Print a file
<code>man</code>	Manual page
<code>help</code>	Help on shell built-ins

<code>what is / apropos</code>	One-line description / keyword search
<code>bc</code>	Arbitrary-precision calculator
<code>expr</code>	Evaluate integer expression
<code>factor</code>	Prime factorisation
<code>units</code>	Unit conversion
<code>write</code>	Message one logged-in user
<code>wall</code>	Broadcast to all users
<code>mail</code>	Send/read email
<code>mesg</code>	Allow/deny messages to your terminal
<code>who</code>	All logged-in users
<code>who am i</code>	Your own login line
<code>whoami</code>	Your effective username
<code>tty</code>	Your terminal device file
<code>date</code>	Show/set date and time
<code>cal</code>	Display calendar
<code>uname</code>	System/kernel information
<code>uptime</code>	Uptime and load average
<code>ps</code>	Process status
<code>kill</code>	Send a signal to a process
<code>chmod</code>	Change permissions
<code>chown / chgrp</code>	Change owner / group
<code>umask</code>	Default permission mask
<code>su / sudo</code>	Switch user / run as another user
<code>shutdown / halt / reboot</code>	Stop or restart the system
<code>df / du</code>	Disk free / disk usage
<code>tar / gzip</code>	Archive / compress
<code>vi</code>	Visual editor

Likely exam questions (commands)

- [5] Explain any five file-processing commands in UNIX with syntax and an example.
- [5] Differentiate between `cmp`, `diff` and `comm`.
- [5] Explain the `pr` command with any five of its options.
- [5] What are mathematical commands in UNIX? Explain `bc`, `expr`, `factor` and `units`.
- [5] Explain the communication commands `write`, `wall` and `mail`.
- [5] Differentiate between `who`, `who am i` and `whoami`.
- [10] Explain, with syntax and examples, the UNIX file-processing commands `wc`, `head`, `tail`, `cut`, `paste`, `join`, `split`, `sort`, `grep`, `tr`, `comm`, `cmp` and `diff`.
- [15] Describe the essential UNIX commands grouped by purpose (navigation, file management, file processing, formatting/printing, help, mathematical, communication, system information), giving syntax and one example for each.

MCQ traps

Trap	Correct answer
" <code>wc</code> output order?"	lines, words, characters — in that order.
"Default lines shown by <code>head/tail</code> ?"	10 .
"Which command removes a non-empty directory?"	<code>rm -r</code> — <code>rmdir</code> only removes empty ones.
" <code>grep -v</code> ?"	Invert — show non-matching lines (not "verbose").
" <code>cut</code> default delimiter?"	TAB , not space.
"Which requires sorted input?"	<code>comm</code> and <code>join</code> (not <code>diff</code>).
" <code>more</code> vs <code>less</code> ?"	<code>less</code> scrolls both ways and does not load the whole file.
" <code>cal 2026</code> vs <code>cal 9 2026</code> ?"	<code>cal 2026</code> = whole year; month comes first .
" <code>expr 5*3</code> ?"	Error/literal — the <code>*</code> must be escaped: <code>expr 5 * 3</code> .
" <code>echo 10/3 \ bc</code> ?"	3 — integer division unless <code>scale</code> is set.
" <code>tr</code> can take a filename argument?"	No — <code>tr</code> reads only stdin; use <code>tr ... < file</code> .
"Command to broadcast to all users?"	<code>wall</code> .
" <code>mkdir -p</code> ?"	Create parent directories as needed.
"Which command counts only words?"	<code>wc -w</code> .
" <code>sort</code> without <code>-n</code> on numbers?"	Sorts lexicographically — "10" before "9".
" <code>man</code> section for file formats?"	5 .

8. Shell variables

Concept

A shell variable is a name holding a string. What makes shells confusing is that there are **two kinds**, and the difference is about *inheritance*:

- A **local (shell) variable** exists only in the current shell. If that shell starts a child process, the child does **not** see it.
- An **environment variable** has been **exported**. Every child process inherits a *copy*.

The classic exam sentence: **a child can never modify its parent's environment** — it only gets a copy. That is precisely why a script that does `cd /tmp` leaves your shell where it was, and why you must `source script` (or `. script`) to run it in the *current* shell instead of a child.

Rules

- Assignment: `name=value` — **no spaces around =**. `name = value` is an error (the shell reads `name` as a command). This is an MCQ.
- Reference: `$name` or `${name}`. Braces are needed when adjacent text would be ambiguous: `${file}_backup`.
- Quoting matters enormously:

Quoting	Behaviour	Example with <code>x=5</code>
<code>"double"</code>	Variables and command substitution are expanded	<code>echo "\$x" → 5</code>
<code>'single'</code>	Everything literal	<code>echo '\$x' → \$x</code>
<code>`backquote`</code> or <code>\$()</code>	Command substitution — replaced by the command's output	<code>d=\$(date)</code>
<code>\</code>	Escape the next character	<code>echo \ \$x → \$x</code>

- Export: `export PATH=$PATH:/opt/bin` makes it environmental. `env` or `printenv` lists environment variables; `set` lists **all** variables including local ones and functions; `unset name` removes one; `readonly name` makes it immutable.

Standard environment variables

Variable	Meaning
PATH	Colon-separated list of directories searched for commands. <code>/usr/local/bin:/usr/bin:/bin</code> . A . in PATH is a security risk — an attacker leaving a malicious <code>ls</code> in a directory you <code>cd</code> into would get it run
HOME	Your home directory; the target of a bare <code>cd</code>

PS1	Primary prompt string (default <code>\$</code> or <code>\u@\h:\w\</code>)
PS2	Secondary/continuation prompt (default <code>></code>)
SHELL	Path of your login shell
USER / LOGNAME	Your username
PWD / OLDPWD	Current / previous working directory
TERM	Terminal type (<code>xterm</code> , <code>vt100</code>) — used by <code>vi</code> , <code>clear</code>
MAIL	Path to your mailbox
IFS	Internal Field Separator — default space, tab, newline; how the shell splits words
TZ	Time zone
EDITOR / VISUAL	Default editor
HISTSIZE / HISTFILE	History length and file (<code>~/.bash_history</code>)
LANG / LC_ALL	Locale

Special (positional and status) variables — memorise this table

Variable	Meaning
\$0	Name of the script itself
\$1 ... \$9	Positional parameters (arguments). Beyond 9 use <code>\${10}</code>
\$#	Number of arguments passed
\$*	All arguments as a single word ("a b c")
\$@	All arguments as separate words ("a" "b" "c") — safer in loops
\$?	Exit status of the last command : 0 = success , non-zero = failure
\$\$	PID of the current shell
\$_	PID of the last background command
\$-	Current shell option flags
shift	Discards <code>\$1</code> and shifts the rest down

\$? = 0 means success is counter-intuitive and therefore examined every time. Remember:

there is one way to succeed and many ways to fail, so 0 is reserved for success.

Likely exam questions

- [5] What are shell variables? Distinguish between local and environment variables.
- [5] Explain any five environment variables in UNIX.
- [5] Explain the special shell variables `$0`, `$#`, `$?`, `$$` and `$*`.
- [10] Explain shell variables in UNIX: types, assignment and referencing rules, quoting, the `export` mechanism, standard environment variables and special variables, with examples.

MCQ traps

Trap	Correct answer
" <code>x = 5</code> is valid?"	No — no spaces around <code>=</code> .
" <code>\$?</code> returns 0 means?"	Success .
" <code>\$\$</code> is?"	PID of current shell (<code>\$!</code> is the last background PID).
" <code>\$#</code> ?"	Number of arguments, not including <code>\$0</code> .
"Difference between <code>\$*</code> and <code>\$@</code> ?"	<code>\$*</code> = one word; <code>\$@</code> = separate words (when quoted).
"Which command lists only exported variables?"	<code>env</code> / <code>printenv</code> (<code>set</code> lists all).
"Variable inside single quotes?"	Not expanded .
"Can a child change the parent's environment?"	No — it gets a copy.

9. Redirection, pipes and filters

Concept

Every UNIX process starts life with **three open files**, identified by small integers called **file descriptors**:

FD	Name	Default connection
0	stdin — standard input	Keyboard
1	stdout — standard output	Screen/terminal
2	stderr — standard error	Screen/terminal

Because they are just file descriptors, the shell can quietly reconnect them to files or to each other **before** the program starts. The program never knows and never has to care —

this is the whole trick behind pipes and redirection, and it is the sentence to open a 10-mark answer with.

Redirection operators

Operator	Meaning
<code>></code>	Redirect stdout to a file, overwriting (creates if absent, truncates if present)
<code>>></code>	Redirect stdout to a file, appending
<code><</code>	Take stdin from a file
<code>2></code>	Redirect stderr
<code>2>></code>	Append stderr
<code>&></code> or <code>>file 2>&1</code>	Redirect both stdout and stderr to the same file
<code>2>&1</code>	Make stderr go wherever stdout is currently going
<code>1>&2</code>	Send stdout to stderr
<code><<EOF</code>	Here-document — feed the following inline lines as stdin until <code>EOF</code>
<code><<<"text"</code>	Here-string (bash)
<code>>/dev/null 2>&1</code>	Discard all output — <code>/dev/null</code> is the "bit bucket"
<code>\ </code>	Pipe — connect stdout of the left command to stdin of the right
<code>\ &</code>	Pipe both stdout and stderr (bash)

Order matters, and this is an examiner's favourite: `cmd > file 2>&1` sends **both** to the file, because stdout is redirected first and then stderr is pointed at the same place. But `cmd 2>&1 > file` sends **stderr to the terminal** and stdout to the file, because when `2>&1` executes, stdout was still the terminal. Read right-to-left in terms of effect.

Pipes

A **pipe** (`|`) is an in-memory buffer connecting two processes. Both processes run **concurrently**, not one after the other — the second starts consuming as the first produces.

```
who | wc -l          # how many users are logged in
ls -l | grep "^d"    # list only directories
cat f | sort | uniq -c | sort -rn | head -5
```

Pipe vs redirection (5-mark comparison):

	Pipe <code>\ </code>	Redirection <code>> <</code>
--	----------------------	------------------------------------

Connects	Two processes	A process and a file
Intermediate file	None (memory buffer)	Yes
Concurrency	Both processes run simultaneously	Sequential
Direction	Always stdout → stdin	Either

tee solves the "I want to see it *and* save it" problem: `ls -l | tee list.txt | wc -l`
 writes the listing to `list.txt` while passing it on to `wc`.

Filters

A **filter** reads stdin, transforms, writes stdout. That definition (worth stating verbatim) is what makes a command usable in the middle of a pipeline. Standard filters: `cat`, `head`, `tail`, `sort`, `uniq`, `grep`, `tr`, `cut`, `paste`, `wc`, `sed`, `awk`, `tee`, `more`, `less`, `nl`, `pr`, `rev`, `fmt`. Non-filters: `ls`, `cp`, `mv`, `rm`, `mkdir`, `date`, `who` (they generate or act but do not read stdin).

Wildcards / metacharacters (shell globbing)

Expansion is done by **the shell, not the command** — by the time `ls` runs, it has already been handed the expanded list of names. State this; it is a mark.

Metachar	Matches
<code>*</code>	Any string of zero or more characters (but not a leading dot)
<code>?</code>	Exactly one character
<code>[abc]</code>	Any one of the listed characters
<code>[a-z]</code> , <code>[0-9]</code>	Any one character in the range
<code>[!abc]</code> or <code>[^abc]</code>	Any one character not listed
<code>{a,b,c}</code>	Brace expansion — generates <code>a b c</code> (not a wildcard; no file need exist)
<code>~</code>	Home directory
<code>;</code>	Command separator (run sequentially regardless of success)
<code>&&</code> / <code>\ \ </code>	Run next only if previous succeeded / failed
<code>&</code>	Run in background
<code>\</code>	Escape

Examples: `ls *.txt`; `rm file?.log`; `ls [a-c]*`; `cp *.{jpg,png} images/`.

Likely exam questions

- [5] What are standard input, standard output and standard error? Give their file descriptor numbers.
- [5] Explain input and output redirection in UNIX with examples.
- [5] What is a pipe? Differentiate between a pipe and redirection.
- [5] Define a filter. Name any six filter commands.
- [10] Explain redirection, pipes and filters in UNIX with suitable examples, including how to redirect standard error and how `tee` works.

MCQ traps

Trap	Correct answer
"File descriptor of stderr?"	2 (stdin 0, stdout 1).
" <code>></code> on an existing file?"	Overwrites/truncates it. Use <code>>></code> to append.
"Which discards output?"	<code>> /dev/null</code> .
"Who expands <code>*</code> ?"	The shell , before the command runs.
"Does <code>*</code> match hidden files?"	No — not a leading dot.
"Does <code>ls > f</code> redirect errors too?"	No — only stdout. Need <code>2>&1</code> .
"In a pipe, do the commands run sequentially?"	No — concurrently.

10. Basic shell scripting

Concept

A shell script is a plain text file of commands. Two things make it a program rather than a list: the **shebang** and the **execute bit**.

```
#!/bin/bash
# The first two characters must be #! and it must be line 1.
# The kernel reads them and runs /bin/bash with this file as its argument.
```

Then `chmod +x script.sh` and run it as `./script.sh` (the `./` is needed because `.` is normally **not** in `PATH`, deliberately). Alternatively `bash script.sh` (shebang ignored) or `. script.sh` / `source script.sh` (runs in the **current** shell, so variable and `cd` changes persist).

Constructs

```

# --- variables and input ---
name="Ravi"
read -p "Enter your age: " age

# --- conditionals ---
if [ $age -ge 18 ]; then
    echo "$name is an adult"
elif [ $age -ge 13 ]; then
    echo "$name is a teenager"
else
    echo "$name is a child"
fi

# --- case ---
case $choice in
    1) echo "One" ;;
    2|3) echo "Two or three" ;;
    *) echo "Invalid" ;;
esac

# --- loops ---
for i in 1 2 3 4 5; do echo "Number $i"; done
for f in *.txt; do echo "Found $f"; done
for ((i=1; i<=5; i++)); do echo $i; done

count=1
while [ $count -le 5 ]; do
    echo $count
    count=$((count+1))
done

until [ $count -gt 10 ]; do count=$((count+1)); done

# --- function ---
greet() {
    echo "Hello, $1"
    return 0
}
greet "World"

```

Test operators — a table you should be able to reproduce

Numeric	Meaning	String	Meaning	File	Meaning
<code>-eq</code>	equal	<code>=</code>	equal	<code>-e</code>	exists

<code>-ne</code>	not equal	<code>!=</code>	not equal	<code>-f</code>	is a regular file
<code>-gt</code>	greater than	<code>-z</code>	zero length	<code>-d</code>	is a directory
<code>-ge</code>	$\geq$	<code>-n</code>	non-zero length	<code>-r -w -x</code>	readable/writable/executable
<code>-lt</code>	less than	<code>< ></code>	lexical order	<code>-s</code>	size greater than zero
<code>-le</code>	$\leq$			<code>-L</code>	is a symbolic link

Combine with `-a` (and), `-o` (or), `!` (not) inside `[ ]`, or `&&/||` between `[ [ ] ]`.

Spaces inside the brackets are mandatory: `[ $a -eq $b ]`, never `[$a -eq $b]` — `[` is literally a command (`/usr/bin/[`), so it needs to be a separate word. That is a superb MCQ explanation to give.

Worked script examples

1. Sum of command-line arguments and argument checking

```
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "Usage: $0 num1 num2 ..."
    exit 1
fi
sum=0
for n in "$@"; do
    sum=$((sum + n))
done
echo "Number of arguments: $#"
```

```
echo "Sum = $sum"
```

2. Check whether a file exists and report its type

```
#!/bin/bash
read -p "Enter filename: " f
if [ ! -e "$f" ]; then
    echo "$f does not exist"; exit 1
elif [ -d "$f" ]; then
    echo "$f is a directory"
elif [ -f "$f" ]; then
    echo "$f is a regular file, $(wc -l < "$f") lines"
fi
[ -r "$f" ] && echo "readable"
[ -w "$f" ] && echo "writable"
```

3. Factorial (demonstrates a loop and arithmetic)

```
#!/bin/bash
```

```

read -p "Enter a number: " n
fact=1
for ((i=2; i<=n; i++)); do
    fact=$((fact * i))
done
echo "Factorial of $n is $fact"

```

4. Menu-driven script (a very common exam ask)

```

#!/bin/bash
while true; do
    echo "1. Date  2. Users  3. Directory listing  4. Exit"
    read -p "Choice: " ch
    case $ch in
        1) date ;;
        2) who ;;
        3) ls -l ;;
        4) exit 0 ;;
        *) echo "Invalid choice" ;;
    esac
done

```

Likely exam questions

- **[5]** What is a shell script? How is it created and executed?
- **[5]** Write a shell script to find the largest of three numbers.
- **[5]** Explain the decision-making statements available in the shell.
- **[10]** Write a shell script that accepts a filename and reports whether it exists, its type, its permissions and its line count. Explain each construct used.
- **[10]** Explain shell programming: variables, control structures (`if`, `case`, `for`, `while`, `until`) and functions, with a complete worked script.
- **[15]** Explain shell scripting in UNIX, covering the shebang, execution methods, variables, positional parameters, test operators, all control structures and functions. Illustrate with at least two complete scripts.

MCQ traps

Trap	Correct answer
" <code>#!</code> is called?"	Shebang / hashbang; must be line 1 .
" <code>#</code> elsewhere?"	A comment .
"Why <code>./script</code> ?"	<code>.</code> is not in <code>PATH</code> by default.
"Difference between <code>bash s.sh</code> and <code>. s.sh</code> ?"	The latter runs in the current shell (changes persist).

"[\$a -eq \$b] fails because?"	Missing space before <code>]; [</code> is a command.
"String equality operator?"	<code>=</code> (or <code>==</code> in bash), not <code>-eq</code> (that is numeric).
" <code>-z</code> tests?"	String is empty .
"How to exit with an error status?"	<code>exit 1</code> (non-zero).

11. Process management

Concept

A **program** is a passive file on disk. A **process** is a program in execution — it has an address space, a program counter, registers, open files, and an entry in the kernel's process table. Every process has:

- a **PID** (process ID), unique;
- a **PPID** (parent PID) — because in UNIX **every process except `init` is created by another process**;
- an owner (real and effective UID/GID), a priority, a current directory, and a controlling terminal.

How processes are born. UNIX does it in a way that surprises people: there is no "run this program" call. Instead:

1. `fork()` — the parent makes an almost-exact **duplicate** of itself. The duplicate is the child. `fork()` returns **0 in the child** and the **child's PID in the parent** — this is how each knows which one it is. Returns `-1` on failure.
2. `exec()` — the child then **overlays itself** with the new program. Its PID does not change; only its memory image does. Hence "**fork and exec**".
3. `wait()` — the parent suspends until the child finishes and collects its exit status.
4. `exit()` — the child terminates, returning a status.

Two consequences worth stating:

- A **zombie** is a process that has terminated but whose parent has not yet called `wait()`, so its exit status still occupies a process-table slot. It uses no memory or CPU — it is just an entry. Shown as `Z` or `<defunct>`. You cannot kill a zombie; you kill or fix the **parent**.
- An **orphan** is a process whose parent died first. The kernel immediately **re-parents it to `init` (PID 1)**, which reaps it properly. Orphans are harmless; zombies are the leak.

Process states

State	ps code	Meaning
Running / runnable	R	Executing or on the run queue
Interruptible sleep	S	Waiting for an event (I/O, input)
Uninterruptible sleep	D	Waiting on disk I/O; cannot be killed until it returns
Stopped	T	Suspended (Ctrl+Z or SIGSTOP)
Zombie	Z	Terminated, not yet reaped

Commands

Command	Purpose	Options
ps	Process status	ps current terminal only; ps -ef (System V) all processes full format; ps aux (BSD) all with CPU/memory; ps -u ravi ; ps -l long, shows priority/nice
top / htop	Live, sorted, updating process display	top, then k kill, q quit
pstree	Processes as a tree showing parentage	pstree -p
pgrep / pkill	Find / signal processes by name	pkill -9 firefox
kill	Send a signal (default SIGTERM 15)	kill 1234 ; kill -9 1234 SIGKILL; kill -l list signals
killall	Kill by name	killall httpd
&	Run a command in the background	sleep 100 &
jobs	List background jobs of this shell	jobs -l
fg / bg	Bring job to foreground / resume in background	fg %1 , bg %1
nohup	Run immune to hangup — survives logout; output to nohup.out	nohup ./long.sh &
nice / renice	Start / change scheduling priority. Nice value -20 (highest priority) to +19 (lowest) ; only root may lower it below 0	nice -n 10 cmd ; renice -5 -p 1234
wait	Wait for background jobs to finish	wait
sleep	Pause n seconds	sleep 5

<code>at / batch</code>	Run a command once at a given time	<code>at 5pm</code>
<code>cron / crontab</code>	Run commands on a repeating schedule	<code>crontab -e, crontab -l</code>
<code>time</code>	Report real/user/sys time for a command	<code>time ./prog</code>
<code>lsof</code>	List open files (and which process holds them)	<code>lsof -p 1234</code>
<code>strace</code>	Trace system calls	<code>strace -p 1234</code>

Cron field order (a reliable MCQ): `minute hour day-of-month month day-of-week command`
— e.g. `30 2 * * 1 /backup.sh` runs at 02:30 every Monday. Minute 0–59, hour 0–23, DOM 1–31, month 1–12, DOW 0–7 (0 and 7 both = Sunday).

Important signals

Signal	Number	Meaning	Catchable?
SIGHUP	1	Hangup — terminal closed; conventionally makes daemons reload config	Yes
SIGINT	2	Interrupt — Ctrl+C	Yes
SIGQUIT	3	Quit with core dump — Ctrl+\	Yes
SIGKILL	9	Kill immediately — cannot be caught, blocked or ignored	No
SIGSEGV	11	Segmentation fault	Yes
SIGTERM	15	Terminate politely — the default of <code>kill</code> ; lets the program clean up	Yes
SIGSTOP	19	Suspend — cannot be caught	No
SIGTSTP	20	Terminal stop — Ctrl+Z	Yes
SIGCONT	18	Continue a stopped process	Yes
SIGCHLD	17	Child stopped or terminated	Yes

Always try `kill -15` before `kill -9`. SIGTERM lets a program flush buffers, close files and remove lock files; SIGKILL is handled by the kernel and gives the process no chance at all, so it can corrupt data. Saying this is worth a mark.

Likely exam questions

- **[5]** What is a process? Differentiate between a program and a process.
- **[5]** Explain `fork()` and `exec()` in UNIX process creation.

- **[5]** What are zombie and orphan processes?
- **[5]** Explain foreground and background processing in UNIX with examples.
- **[5]** Explain the `kill` command. Differentiate between signals 9 and 15.
- **[10]** Explain process management in UNIX: process attributes, states, creation via `fork/exec/wait/exit`, and the commands used to monitor and control processes.
- **[15]** Describe the UNIX process model in detail. Include the process table, PID/PPID relationships, the fork-exec mechanism with a diagram, process states with a state-transition diagram, zombies and orphans, signals, job control and scheduling priority (`nice`).

MCQ traps

Trap	Correct answer
" <code>fork()</code> returns what in the child?"	0 . In the parent: the child's PID .
"Does <code>exec</code> create a new process?"	No — it replaces the current process image; PID unchanged.
"PID 1?"	<code>init/systemd</code> .
"Which signal cannot be caught?"	SIGKILL (9) and SIGSTOP (19) .
"Default signal of <code>kill</code> ?"	SIGTERM (15) , not 9.
"Ctrl+C sends?"	SIGINT (2) . Ctrl+Z → SIGTSTP (20) .
"A zombie consumes memory?"	No — only a process-table entry.
"Orphan is adopted by?"	<code>init</code> (PID 1).
"Highest priority nice value?"	-20 (lowest is +19). Lower number = higher priority.
"Which command runs a job after logout?"	<code>nohup</code> .

12. Last-week revision sheet

Twelve facts most likely to appear as one-mark MCQs:

1. UNIX — Ken Thompson, 1969, PDP-7, assembly; **rewritten in C in 1973** by Ritchie.
2. Linux kernel — **Linus Torvalds, 1991**; UNIX-like, POSIX-compliant, not UNIX-derived.
3. Shell = user program, **not** part of the kernel; interface via **system calls**.
4. File system regions: **boot block, super block, inode block, data block**.
5. Inode holds everything **except the file name** (which is in the directory entry).
6. `r=4`, `w=2`, `x=1`. **umask 022** → **files 644, directories 755**.
7. On a **directory**: `x` = traverse, `w` = create/delete entries, `r` = list names.

8. **SUID 4000, SGID 2000, sticky 1000**; `/tmp` is `drwxrwxrwt`.
9. `wc` prints **lines, words, characters**; `head/tail` default to **10** lines.
10. `comm` and `join` need **sorted** input; `cmp` gives the **first** difference.
11. **`$?` = 0 means success**; `$$` = current shell PID; `$#` = argument count.
12. **SIGKILL 9 cannot be caught**; `kill` defaults to **SIGTERM 15**; PID 1 is `init`.

Descriptive-paper strategy for this topic. Section A (5 marks) will almost certainly offer at least one UNIX question in the Diploma paper — take it, they are the cheapest marks on the paper. For Section B/C, the reliably examinable long answers are: (a) UNIX architecture with diagram, (b) the file system with the four regions and inodes, (c) permissions with worked `chmod/umask` arithmetic, (d) the boot sequence, (e) a grouped command catalogue, (f) shell scripting with a complete script. Prepare a **clean, fast, labelled diagram** for (a), (b) and (d) — a good diagram earns 3–4 of the 10 or 15 marks on its own, and it takes thirty seconds.